Entornos de desarrollo

None

Cristian Garcia

None

entornosdedesarrollo

1. Bienvenid@ al módulo de Entornos de Desarrollo 3
1.1 Bolsa de proyectos 3
2. Sprint 1. Ecosistema de desarrollo, marco ágil y repositorio digital 4
2.1 Semana 1. Ciclo de vida (SDLC) y taller IDE + JDK 4
2.2 Semana 2. Estados de Git, diff visual y commits 12
2.3 Semana 3. Cierre de sprint e higiene (.gitignore) 18
3. Unidad 2 24
3.1 Semana 4. Código fuente, bytecode y JVM 24
3.2 Semana 5. Construcción y proyectos con Maven 29
3.3 Semana 6. Gestión de dependencias y cierre 34
4. Sprint 3. El ciclo del error: pruebas y depuración 42
4.1 Semana 7. Depuracion interactiva e inspección de memoria 42
4.2 Semana 8. Clases de equivalencia y cobertura 47
4.3 Semana 9. Complejidad ciclomática y regresión 53

1. Bienvenid@ al módulo de Entornos de Desarrollo

El equipo de estudiantes debe elegir uno de los proyectos que se muestran a continuación.

1.1 Bolsa de proyectos

1.1.1 Ciberseguridad

1. Simulador de phishing — juego interactivo de detección de mensajes fraudulentos; clasificación de mensajes por tipología (Email, SMS, Red Social) con pistas contextuales, gestión de usuarios/sesiones, estadísticas de aciertos e historial de partidas.
2. Bóveda de contraseñas — gestor de credenciales con evaluación de fortaleza mediante políticas de seguridad (Básica, Estricta, Corporativa); control de caducidad, filtrado de cuentas guardadas y persistencia del almacén seguro.

1.1.2 Python

1. Calculadora paso a paso — evaluador de expresiones matemáticas con desglose paso a paso; tokenización, precedencia y cálculo con operadores polimórficos, interfaz con visor del proceso de resolución e histórico de cálculos.
2. Generador de datos de prueba — herramienta de síntesis de datasets sintéticos a partir de plantillas; campos polimórficos (Texto, Número, Fecha, Email), generación de lotes de registros aleatorios y exportación a ficheros CSV.

1.1.3 Videojuegos y realidad virtual

1. Aventura conversacional — motor de narrativa interactiva ramificada con toma de decisiones; gestión de escenas, inventario del jugador, interacción con diferentes tipos de personajes (Aliado, Enemigo, Neutral) y guardado del estado de la partida.
2. Simulador de físicas 2D — entorno de simulación cinemática con detección de colisiones y rebotes; escenario visual con múltiples bolas simultáneas de comportamiento polimórfico (Normal, Pesada, Rebote especial) y exportación de estadísticas de la simulación.

1.1.4 Productos software en contenedores

1. Reparto de contenedores en nodos — planificador de asignación de carga en clúster según capacidad de CPU y memoria; gestión de nodos especializados (Alta Memoria, Alta CPU), interfaz gráfica con barras de ocupación y persistencia de configuraciones de reparto.
2. Catálogo de servicios + generador de config — asistente paso a paso (wizard) para la definición y parametrización de servicios (Web, BD, Worker); generación polimórfica de ficheros de configuración, catálogo de servicios e historial de despliegues.

1.1.5 Inteligencia artificial y big data

1. Motor de recomendación — sistema de recomendación basado en afinidad entre gustos y catálogos de ítems categorizados (Película, Música, Libro); cálculo de puntuaciones de coincidencia por etiquetas, gestión de perfiles y persistencia de valoraciones.
2. Analizador de reseñas — analizador de sentimiento léxico sobre comentarios; procesamiento de palabras clave mediante diccionarios específicos según el tipo de producto, estadísticas agregadas por producto y persistencia del catálogo de reseñas.

1.1.6 Recursos y servicios en la nube

1. Cotizador cloud — presupuestador interactivo de infraestructura en la nube; cálculo polimórfico de costes según el tipo de recurso (Cómputo, Almacenamiento, Red), visor para montar y comparar presupuestos y persistencia de ofertas para clientes.
2. Dashboard de monitorización — panel de control visual para métricas y telemetría de sistemas; simulación de lecturas de sensores (CPU, Memoria, Red), series temporales, detección de umbrales con disparo de alertas e histórico persistido de incidencias.

2. Sprint 1. Ecosistema de desarrollo, marco ágil y repositorio digital

2.1 Semana 1. Ciclo de vida (SDLC) y taller IDE + JDK

2.1.1 Día 1 - 2 sesiones

Sesión 1. Teoría. Ingeniería del software y herramientas

1. Caso guía en AzaharTech

Son las tres de la tarde del lunes 14 de septiembre de 2026. Las oficinas de **AzaharTech**, consultora de desarrollo de software situada en el distrito tecnológico de Castellón de la Plana, bullen de actividad. Los miembros del equipo de desarrollo asignado a proyectos educativos y de gestión local ocupan sus puestos: **Alba Torres**, desarrolladora especializada en arquitectura de software; **Pau Ferrer**, técnico de desarrollo e integración de sistemas; y la supervisora del equipo, **Laia Claramunt**. Junto a ellos se sienta el nuevo desarrollador junior que acaba de incorporarse a la empresa (el estudiante).

En la pantalla táctil de la sala de reuniones, Laia proyecta una fotografía de la fachada del **IES El Caminàs**, emblemático centro educativo de la ciudad:

«Equipo, bienvenidos al arranque del curso académico. Este trimestre tenemos un proyecto de primer nivel. El equipo directivo del IES El Caminàs nos ha contratado formalmente para digitalizar y automatizar el acceso al recinto escolar. Cada mañana, cientos de estudiantes se agolpan en la entrada, provocando retrasos en el inicio de las clases y obligando a conserjería y a los profesores de guardia a anotar incidencias en listas de papel que tardan horas en llegar a jefatura de estudios.

*Nos han encargado desarrollar un **sistema de control de asistencia mediante códigos QR dinámicos**. La solución mostrará un código QR variable en una gran pantalla situada en el vestíbulo principal, que los estudiantes escanearán desde su teléfono móvil al cruzar la puerta de entrada.*

*Este proyecto del IES El Caminàs será nuestro **caso guía de referencia**: sobre él modelaremos en clase cada decisión técnica y cada arquitectura. Pero, paralelamente, cada uno de vuestros equipos trabajará sobre el proyecto singular que habéis seleccionado de nuestra bolsa de proyectos.*

*Ahora bien: antes de teclear una sola línea de código en un editor, necesitamos aprender a pensar como ingenieros. No vamos a construir un simple script casero; vamos a levantar un **Sistema de Información profesional**. Para que el proyecto no naufrague, hoy debemos entender en profundidad qué etapas componen el ciclo de vida del software, cómo se transforman los lenguajes de programación y cómo preparar nuestro taller de trabajo con las herramientas adecuadas».*

2. De la instrucción aislada al Sistema de Información

En el argot popular es muy común escuchar términos como *programa*, *código*, *aplicación* o *sistema* como si fuesen sinónimos intercambiables. Sin embargo, en el ámbito de la ingeniería del software y el desarrollo de aplicaciones multiplataforma (DAM), establecer con rigor sus fronteras conceptuales es el primer paso obligatorio.

A. PROGRAMA INFORMÁTICO

Un **programa informático** es una secuencia estructurada, lógica y finita de instrucciones escritas en un lenguaje de programación formal, diseñada para que un procesador ejecute una serie de tareas concretas de cálculo o manipulación de datos. Por ejemplo: un algoritmo que recibe la hora actual y la hora oficial de entrada y calcula la diferencia en minutos es un programa informático.

B. SISTEMA DE INFORMACIÓN (SI)

Un **Sistema de Información (SI)** es un ecosistema mucho más complejo, amplio y multidimensional. Se define como un conjunto organizado de elementos interactivos que recopilan, procesan, almacenan, aseguran y distribuyen datos para apoyar la toma de decisiones, la coordinación, el control y la operativa diaria de una organización o negocio.

Un Sistema de Información no se limita al código fuente; está formado por **cinco componentes indispensables** que deben encajar con precisión:

LOS CINCO COMPONENTES DE UN SISTEMA DE INFORMACIÓN		
Componente	Definición técnica	Aplicación al IES El Caminàs
1. Hardware	Dispositivos físicos, máquinas, terminales, procesadores y redes.	Pantalla del vestíbulo, servidores del centro, teléfonos móviles.
2. Software	Programas, aplicaciones y lógica ejecutada sobre el hardware.	Backend en Java, lectores QR, interfaz gráfica, bases de datos.
3. Datos	Materia prima de la información; registros brutos sin procesar.	Censos de alumnos, matrículas, horas exactas de fichaje, grupos.
4. Personas	Usuarios finales, administradores y personal técnico del sistema.	Alumnado, docentes de guardia, conserjería, equipo directivo.
5. Procesos	Reglas de negocio, protocolos y flujos operativos de la entidad.	Protocolo de entrada: retrasos a partir de las 08:05 h, avisos web.

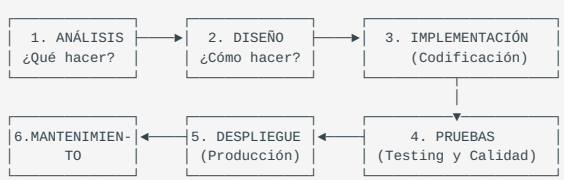
Reflexión para el desarrollador: Como programadores, nosotros producimos directamente el **Software**. Sin embargo, si nuestro código no contempla las limitaciones del **Hardware** (pantallas lentas o móviles sin cámara de calidad), el perfil de las **Personas** que lo utilizarán (estudiantes con prisa o docentes sin formación técnica) o los **Procesos** de la organización, el proyecto fracasará aunque el compilador no devuelva ningún error.

---

3. El Ciclo de Vida del Desarrollo de Software (SDLC)

El software no se fabrica como una pieza física en una cadena de montaje; el software se diseña, se modela, se implementa y evoluciona. La experiencia acumulada durante décadas en la industria informática demostró que lanzarse a programar sin una metodología previa desembocaba invariablemente en proyectos que duplicaban su presupuesto, se entregaban con meses de retraso o no cumplían lo que el cliente necesitaba.

Para garantizar la calidad técnica y la viabilidad económica de cualquier desarrollo, se aplica el **Ciclo de Vida del Desarrollo de Software** (*Software Development Life Cycle* o **SDLC**): un marco de trabajo formal que define las fases secuenciales e iterativas que transcurren desde que surge una necesidad operativa hasta que el software se retira definitivamente del servicio.



FASE 1. ANÁLISIS DE REQUISITOS

Es la fase inicial y más crítica. Su objetivo es responder con absoluta claridad a la pregunta: **¿Qué debe hacer el software?** * Se mantienen entrevistas con el cliente (jefatura de estudios del IES El Caminàs). * Se redactan los **requisitos funcionales** (lo que el sistema debe hacer: *«generar un QR dinámico cada 10 segundos»*) y los **requisitos no funcionales** (restricciones de rendimiento, seguridad o tecnología: *«el sistema debe responder en menos de 1 segundo con 50 accesos simultáneos»*). * *Regla de oro de la ingeniería:* Un error detectado en la fase de análisis cuesta hasta 100 veces menos de solucionar que si se descubre cuando el programa ya está instalado en el cliente.

FASE 2. DISEÑO DE LA ARQUITECTURA

Responde a la pregunta: **¿Cómo lo vamos a construir?** * Se divide el sistema en subsistemas y componentes independientes (módulo de generación visual, módulo de red, módulo de validación matemática). * Se eligen las tecnologías, se diseñan los esquemas de bases de datos, los diagramas de clases y los bocetos de las interfaces de usuario.

FASE 3. IMPLEMENTACIÓN (CODIFICACIÓN)

Es la fase en la que los desarrolladores de AzaharTech traducen los modelos y especificaciones de diseño a código fuente legible utilizando el lenguaje de programación elegido (**Java**). Se aplican estándares de estilo, control de versiones y buenas prácticas de ingeniería.

FASE 4. PRUEBAS Y VERIFICACIÓN (TESTING)

El código escrito no puede entregarse al cliente sin someterse a pruebas rigurosas. * Se ejecutan pruebas para detectar defectos de programación (*bugs*), fallos de seguridad y cuellos de botella de rendimiento. * Se comprueba que el software cumple estrictamente con los requisitos definidos en la Fase 1.

FASE 5. DESPLIEGUE (PUESTA EN PRODUCCIÓN)

El software se instala en el entorno real del cliente: se configuran los servidores del IES El Caminàs, se instala la aplicación que proyectará en la pantalla del vestíbulo y se capacita al personal (profesorado de guardia y conserjería) en el uso de la herramienta.

FASE 6. MANTENIMIENTO Y EVOLUCIÓN

Es la etapa más larga de todo el ciclo de vida (abarca años). Incluye cuatro vertientes de trabajo: * **Mantenimiento correctivo:** Arreglar errores imprevistos que surgen durante el uso diario. * **Mantenimiento adaptativo:** Modificar el software cuando cambia el entorno (por ejemplo, una actualización del sistema operativo de los servidores del centro educativo). * **Mantenimiento perfectivo y evolutivo:** Añadir nuevas funcionalidades solicitadas por el instituto (por ejemplo, permitir que el profesorado fiche también las guardias de patio mediante la misma aplicación).

---

4. Lenguajes de programación, paradigmas y la Máquina Virtual de Java (JVM)

Para comunicarnos con el hardware, los desarrolladores escribimos texto en un archivo informático siguiendo las reglas sintácticas y semánticas de un lenguaje formal.

A. NIVELES DE ABSTRACCIÓN DEL SOFTWARE

1. **Lenguaje máquina (Bajo nivel absoluto):** Es el único lenguaje que los circuitos electrónicos de la CPU comprenden directamente. Está compuesto exclusivamente por secuencias de dígitos binarios (`0` y `1`). Cada modelo de procesador (Intel x86, AMD, ARM) tiene su propio juego de instrucciones máquina incompatible con los demás.
2. **Lenguaje ensamblador (Bajo nivel simbólico):** Sustituye las cadenas binarias por códigos nemotécnicos legibles por humanos (`MOV`, `ADD`, `JMP`), pero sigue estando estrictamente acoplado a un procesador concreto.
3. **Lenguajes de alto nivel (Java, C#, Python):** Permiten al programador expresar algoritmos utilizando palabras en inglés (`class`, `public`, `if`, `while`) y conceptos matemáticos abstractos, independizándose del procesador físico sobre el que se ejecute la aplicación.

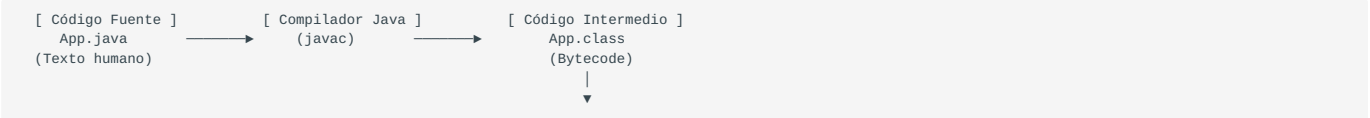
B. PARADIGMAS DE PROGRAMACIÓN

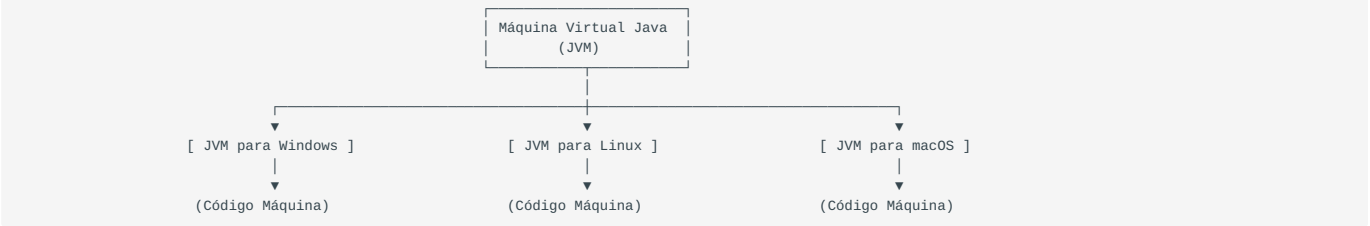
Un paradigma es la filosofía fundamental que determina cómo un desarrollador concibe y estructura la resolución de un problema mediante código: * **Paradigma Imperativo / Estructurado:** El programa es una secuencia ordenada de pasos, instrucciones y llamadas a funciones que modifican el estado de las variables en memoria. * **Paradigma Orientado a Objetos (POO):** El programa se concibe como una red de entidades independientes llamadas **objetos**, que encapsulan tanto sus características o datos internos (**atributos**) como las acciones o funciones que pueden realizar (**métodos**). En el caso de AzaharTech, modelaremos objetos como `Estudiante`, `TerminalQR`, `Fichaje` y `Incidencia`.

C. EL MODELO DE EJECUCIÓN DE JAVA: CÓDIGO FUENTE, BYTECODE Y LA JVM

En los lenguajes tradicionales compilados (como C o C++), el compilador traduce el código fuente directamente a código máquina específico del sistema operativo. Esto obliga a recompilar el programa para Windows, para Linux y para macOS.

Java resolvió este problema introduciendo una capa intermedia de virtualización de software:





1. El desarrollador escribe el código fuente en un archivo con extensión `.java`.
2. El compilador de Java (`javac`) traduce ese archivo a un lenguaje intermedio y universal llamado **Bytecode** (guardado en archivos con extensión `.class`).
3. La **Máquina Virtual de Java (Java Virtual Machine - JVM)** instalada en el ordenador del cliente lee ese Bytecode en tiempo real y lo traduce a instrucciones máquina específicas del procesador local.

> **El principio WORA:** Gracias a la JVM, se cumple el lema fundacional de Java: *«Write Once, Run Anywhere»* (Escribe una vez, ejecuta en cualquier parte). El software que desarrollemos para el IES El Caminàs funcionará exactamente igual en los ordenadores de gestión con Windows que en los servidores web del instituto con Linux.

---

5. Anatomía de un Entorno de Desarrollo Integrado (IDE)

Un programador que pretenda desarrollar software profesional utilizando únicamente un bloc de notas y ejecutando comandos manuales en la terminal perdería hasta el 80 % de su tiempo en tareas mecánicas y propensas a errores.

Un **Entorno de Desarrollo Integrado (IDE, por sus siglas en inglés *Integrated Development Environment*)** es una suite de software que centraliza en una única interfaz gráfica todas las herramientas requeridas para construir, depurar y probar aplicaciones con rapidez y fiabilidad:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    ANATOMÍA DE UN IDE PROFESIONAL (INTELLIJ IDEA)        │
├──────────────────────────────────────────────────────────────────────────┤
│ 1. Editor de código fuente avanzado:                                     │
│    • Resaltado sintáctico de palabras clave, tipos y variables en colores.  │
│    • Autocompletado inteligente (IntelliSense) y plantillas de código vivas. │
│    • Detección de errores léxicos y sintácticos en tiempo real mientras tecleas. │
│                                                                          │
│ 2. Gestor de compilación y construcción automática:                      │
│    • Invocación transparente del compilador con un solo clic o atajo de teclado. │
│    • Gestión de jerarquías de paquetes, librerías externas y dependencias (Maven). │
│                                                                          │
│ 3. Depurador interactivo (Debugger):                                     │
│    • Capacidad de pausar la ejecución del programa en líneas concretas (breakpoints). │
│    • Inspección en vivo del contenido de las variables en la memoria RAM.  │
│                                                                          │
│ 4. Herramientas auxiliares y control de versiones integrado:             │
│    • Conexión visual directa con Git y GitHub sin necesidad de comandos de consola. │
│    • Terminal de comandos integrada, refactorización de código y linters automáticos. │
└──────────────────────────────────────────────────────────────────────────┘
```

En **AzaharTech** hemos estandarizado como IDE corporativo **IntelliJ IDEA Community Edition**, reconocido en el sector como el entorno más potente, ergonómico y extendido para el desarrollo en el ecosistema Java.

Sesión 2. Laboratorio práctico guiado. Puesta a punto del entorno de trabajo

Alba Torres y Pau Ferrer acompañan al estudiante a su puesto de desarrollo:

«Bienvenido a tu puesto de trabajo en AzaharTech. Durante los próximos cincuenta minutos vamos a verificar que tu equipo cuenta con las herramientas oficiales de la empresa: instalaremos el compilador OpenJDK 21, configuraremos IntelliJ IDEA Community y crearemos nuestro primer programa Java para comprobar que todo el engranaje de compilación y ejecución funciona a la perfección».

Paso 1. Instalación y verificación del Java Development Kit (JDK 21)

El **JDK (Java Development Kit)** es el paquete indispensable que incluye el compilador (`javac`), las librerías estándar de Java y la Máquina Virtual (`java`).

1. Descarga e instala la distribución abierta y certificada **OpenJDK 21** (distribución recomendada: *Eclipse Temurin 21 (LTS)* o *Amazon Corretto 21*).
2. Durante el asistente de instalación, asegúrate de marcar las opciones para configurar automáticamente las variables de entorno del sistema (`JAVA_HOME` y adición al `PATH`).
3. Una vez finalizada la instalación, abre un terminal de comandos (*Símbolo del sistema / PowerShell* en Windows o *Terminal* en Linux/macOS).
4. Ejecuta las dos siguientes instrucciones de comprobación:

```
java --version
javac --version
```

5. **Verificación técnica obligatoria:** La consola del sistema debe devolver la información de versión del compilador y de la máquina virtual (por ejemplo: `openjdk 21.0.x 2026-xx-xx`). Si la terminal devuelve un error de tipo *«comando no reconocido»*, significa que la variable de entorno `PATH` no apunta correctamente a la carpeta `bin` del JDK.

Paso 2. Instalación y personalización de IntelliJ IDEA Community Edition

1. Descarga el instalador de **IntelliJ IDEA Community Edition** (versión de software libre).
2. Ejecuta el instalador marcando las siguientes opciones recomendadas:

- *Create Desktop Shortcut* (Crear acceso directo en el escritorio).
- *Add "bin" folder to the PATH* (Añadir carpeta binaria a las variables del sistema).
- *Create Associations: .java* (Asociar archivos fuente Java al entorno).

3. Abre IntelliJ IDEA por primera vez:

- Selecciona el tema visual de interfaz de trabajo (*Dark* o *Light*, según tu comodidad visual).
- En la pantalla de bienvenida, dirígete a **Customize -> Settings...** (en GNU/Linux).
- En la barra de búsqueda escribe `File Encodings` y asegúrate de que **Global Encoding**, **Project Encoding** y **Default encoding for properties files** estén configurados en **`UTF-8`** . Esto evitará problemas con caracteres acentuados o símbolos del valenciano/castellano.

Paso 3. Configuración del SDK (Software Development Kit) en IntelliJ

1. En la ventana principal de IntelliJ IDEA, haz clic en el botón **New Project**.
2. En el panel izquierdo del asistente, selecciona **Java**.
3. En el desplegable denominado **JDK**, comprueba que IntelliJ detecta automáticamente la instalación de **OpenJDK 21**. Si aparece la opción , haz clic en *Add JDK...* y navega hasta el directorio donde se instaló el JDK en el Paso 1 (habitualmente `/usr/lib/jvm/...` en GNU/Linux).
4. Asigna como nombre temporal del proyecto: `VerificacionEntorno` .
5. Pulsa en **Create**.

Paso 4. Creación del primer programa ejecutable

1. En la barra lateral izquierda del IDE, despliega la carpeta del proyecto y localiza el directorio denominado **`src`** (código fuente).
2. Haz clic derecho sobre la carpeta `src` y selecciona: **New -> Java Class**.
3. Escribe como nombre de la clase: `ControlAccesoQR` y presiona *Enter*.

4. Observa cómo IntelliJ genera automáticamente la plantilla básica de la clase: `java public class ControlAccesoQR { }`
5. Escribe dentro de las llaves de la clase el método principal. Puedes utilizar el atajo de teclado inteligente de IntelliJ: escribe `main` y presiona la tecla *Tabulador* o *Enter*. El IDE autocompletará la línea por ti.
6. Dentro del método `main`, utiliza otro atajo inteligente: escribe `sout` y pulsa *Tabulador*. IntelliJ generará automáticamente la instrucción `System.out.println();`.
7. Completa el código para que quede exactamente de la siguiente forma:

```
/**
 * Programa de verificación técnica de puesto de trabajo.
 * AzaharTech Software Consulting - Castellón de la Plana.
 */
public class ControlAccesoQR {
    public static void main(String[] args) {
        System.out.println("================================================");
        System.out.println("   AZAHARTECH - CONSULTORÍA DE SOFTWARE         ");
        System.out.println("   Proyecto: Control de acceso mediante código QR");
        System.out.println("   Cliente:       IES El Caminàs                ");
        System.out.println("================================================");
        System.out.println("Estado del entorno: OpenJDK 21 + IntelliJ OK.");
        System.out.println("Desarrollador/a: Puesto técnico verificado con éxito.");
    }
}
```

Paso 5. Compilación, ejecución y captura de evidencia oficial (Entregable ED-1)

1. Haz clic en el icono del triángulo verde situado a la izquierda de la línea `public class ControlAccesoQR` o pulsa el atajo de teclado universal **`Ctrl + Shift + F10`** (o `Shift + F10`).
2. Observa cómo en la parte inferior del IDE se abre automáticamente la ventana **Run**:

- El IDE invoca a `javac` en segundo plano para compilar el código fuente a Bytecode.
- La JVM ejecuta la clase y la salida formateada aparece en la consola sin advertencias ni errores.

3. **Generación de la evidencia técnica (Captura oficial):**

- Ajusta la ventana de IntelliJ de manera que se visualice claramente el árbol de carpetas a la izquierda, el código fuente en el centro y la consola de ejecución con el mensaje de éxito en la parte inferior.
- Realiza una captura de pantalla completa de tu monitor.
- Guarda provisionalmente la imagen con el nombre exacto **`entorno.png`**. En la próxima sesión (Sesión 2 del viernes), crearemos la jerarquía de carpetas oficial y la subiremos al repositorio remoto de GitHub.

2.1.2 Día 2 - 1 sesión

Teoría. El marco ágil y el repositorio digital

1. Caso guía AzaharTech

Es viernes por la tarde en **AzaharTech**. En las pantallas de los puestos de trabajo, los estudiantes tienen abierto IntelliJ IDEA con la clase `ControlAccesoQR.java` compilada el lunes anterior y la captura `entorno.png` guardada en el escritorio.

Laia Claramunt inicia la reunión de los viernes:

«El lunes dejamos nuestro taller físico a punto. Pero en la ingeniería del software actual, trabajar en un ordenador aislado no sirve de nada si el código no está versionado, protegido y compartido de forma transparente. Además, el IES El Caminàs necesita saber cómo vamos a avanzar semana a semana.

*En AzaharTech no utilizamos el viejo modelo en cascada donde el cliente no ve nada durante meses. Aplicamos **Scrum**: dividimos el trabajo en **sprints de 3 semanas**, y al final de cada sprint entregamos software que funciona. Hoy aprenderemos qué es Scrum y crearemos vuestro **primer repositorio en GitHub**, configurando la estructura de carpetas corporativa oficial que mantendremos durante todo el curso».*

---

2. Fundamentos de Scrum y el Repositorio Digital

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          EL FLUJO DE TRABAJO EN SCRUM                    │
├──────────────────────────┬──────────────────────────┬────────────────────┤
│ 1. Product Backlog       │ 2. Sprint Backlog        │ 3. Incremento de Software │
├──────────────────────────┼──────────────────────────┼────────────────────┤
│ Lista global priorizada  │ Subconjunto de tareas que │ Software 100% operativo,  │
│ de requisitos del cliente│ el equipo se compromete a │ probado y potencialmente  │
│ (IES El Caminàs).        │ terminar en estas 3 semanas.│ desplegable en el cliente. │
└──────────────────────────┴──────────────────────────┴────────────────────┘
```

A. POR QUÉ SCRUM FRENTE AL MODELO EN CASCADA

En los proyectos de software, los requisitos del cliente cambian con frecuencia. El modelo tradicional (*en cascada*) exigía cerrar todos los documentos al principio; si el cliente cambiaba de idea meses después, el proyecto fracasaba.

Scrum es un marco de trabajo ágil e iterativo: * El tiempo se divide en bloques fijos llamados **Sprints** (en nuestro curso, de **3 semanas**). * En cada sprint se diseña, se programa, se prueba y se entrega una parte terminada del sistema (**Incremento**). * **Los roles de AzaharTech:** * **Product Owner:** Representa las necesidades del cliente (equipo directivo del IES El Caminàs). * **Scrum Master (Laia Claramunt):** Vela por la metodología, elimina bloqueos y guía al equipo. * **Developers (Alba, Pau y el estudiante):** Profesionales responsables del diseño técnico, código y pruebas.

B. EL REPOSITORIO REMOTO EN GITHUB COMO CANAL OFICIAL DE ENTREGAS (RA4.H)

Un **repositorio** es una base de datos gestionada por Git que almacena el historial completo de cambios de un proyecto. * **Repositorio local,** Se almacena en el disco duro del estudiante. * **Repositorio remoto (GitHub):** Se almacena en los servidores en la nube de GitHub. Funciona como el **buzón oficial de entregas**, garantizando la autoría de cada línea de código y permitiendo al docente calificar el avance en tiempo real.

Laboratorio práctico guiado. Estructura corporativa y primer push

Alba Torres y Pau Ferrer proyectan sus terminales:

*«Vamos a crear la estructura oficial de carpetas de AzaharTech en vuestros puestos. Moveremos la captura de pantalla del lunes a su carpeta definitiva, configuraremos el archivo `.gitignore` para no subir basura y haremos nuestro primer **commit y push** a GitHub».*

Paso 1. Creación de la estructura oficial de carpetas corporativa

1. Abre el explorador de archivos de tu sistema operativo o la terminal en tu directorio de trabajo personal.
2. Crea la jerarquía oficial de carpetas establecida para el curso:

```
azahartech/
└── nombre-equipo/                    <-- Nombre de tu equipo asignado desde el día 1
    └── apellidos-nombre/             <-- Tus apellidos y nombre (sin espacios ni acentos)
        ├── README.md                 <-- Archivo de portada (lo editaremos en la Semana 2)
        ├── .gitignore                <-- Archivo de exclusión de temporales
        ├── ed/                       <-- Módulo Entornos de Desarrollo
        │   └── docs/
        │       └── entorno.png       <-- Mueve aquí la captura guardada el lunes
        ├── pr/                       <-- Módulo Programación
        │   ├── pseudocodigo/
```

```
        │   └── src/
        └── pi/                        <-- Módulo Proyecto Intermodular
            └── docs/
```

1. Mueve el archivo `entorno.png` (generado en la Sesión 1) dentro del directorio recién creado: `ed/docs/entorno.png`.

Paso 2. Creación del archivo de exclusiones `.gitignore`

En los proyectos de desarrollo profesional **nunca se suben a GitHub los archivos compilados ni las carpetas privadas del IDE**.

1. Dentro de tu carpeta personal (`azahartech/nombre-equipo/apellidos-nombre/`), crea un archivo de texto con el nombre exacto `.gitignore` (con el punto al principio y sin extensión).
2. Ábrelo con IntelliJ o un editor de texto y añade las siguientes reglas de exclusión universales:

```
# Exclusiones de IntelliJ IDEA
.idea/
*.iml
out/

# Exclusiones de compilación y Maven
target/
*.class

# Archivos del sistema operativo
.DS_Store
Thumbs.db
```

Paso 3. Vinculación de IntelliJ con tu cuenta de GitHub

1. Abre IntelliJ IDEA.
2. Ve al menú: **File -> Settings** (en Windows/Linux) o **IntelliJ IDEA -> Preferences** (en macOS).
3. En el buscador escribe **GitHub** (ubicado en *Version Control -> GitHub*).
4. Haz clic en el botón **+** (*Add account*) y selecciona **Log In via GitHub...**.
5. Se abrirá tu navegador web; autoriza la vinculación con JetBrains. Una vez aceptado, tu usuario y avatar aparecerán dentro de IntelliJ.

Paso 4. Inicialización del repositorio local y primer Commit convencional

1. En el menú superior de IntelliJ IDEA, selecciona: **Git -> Create Git Repository...** (o *VCS -> Enable Version Control Integration*).
2. Selecciona la carpeta raíz de tu espacio personal (`azahartech/nombre-equipo/apellidos-nombre/`) y pulsa **OK**.
3. Abre el panel lateral **Commit** (`Alt + 0` o `Ctrl + K`):

- Verás los archivos que acabas de estructurar (`.gitignore` , `ed/docs/entorno.png` , etc.).
- Marca la casilla para incluir todos los archivos (*Stage*).

4. En el campo de mensaje de commit, escribe siguiendo el estándar formal de la industria:

```
feat: inicializar estructura corporativa oficial y verificar entorno con OpenJDK 21
```

5. Haz clic en el botón **Commit**.

Paso 5. Publicación en GitHub (*Push*) y verificación web

1. En el menú superior de IntelliJ, selecciona: **Git -> GitHub -> Share Project on GitHub**.

2. Configura los parámetros de publicación:
- **Repository Name:** `DAM-AzaharTech-Proyecto-TuNombre` (sustituyendo *TuNombre* por tu nombre real).
- **Private:** Desmárcalo (debe ser público para que el docente pueda evaluarlo).
- **Remote:** `origin`.
- **Description:** *Repositorio corporativo de AzaharTech - 1.º DAM.*
3. Haz clic en **Share**. IntelliJ creará el repositorio remoto en los servidores de GitHub y subirá todos los archivos automáticamente.
4. Abre tu navegador web, entra en tu perfil de GitHub y accede al nuevo repositorio:
- Comprueba que la carpeta `ed/docs/entorno.png` está subida y se visualiza la imagen correctamente.
- Comprueba que el mensaje del commit aparece reflejado con el prefijo `feat:`.

2.2 Semana 2. Estados de Git, diff visual y commits

2.2.1 Día 3 - 2 sesiones

Sesión 1. Teoría. La mecánica interna de Git y la trazabilidad

1. Caso guía en AzaharTech. El caos de los «cambios varios» en AzaharTech

Es lunes por la tarde en **AzaharTech**. En la pantalla de un ordenador de desarrollo, **Laia Claramunt** tiene abierto el panel de actividad de GitHub del proyecto del **IES El Caminàs**. Mientras revisa los registros del fin de semana, frunce el ceño.

Llama a **Pau Ferrer**, a **Alba Torres** y al estudiante:

«Mirad el registro de actividad del viernes a última hora. Hay tres confirmaciones seguidas de Pau con los siguientes mensajes: 'cambios', 'subiendo cosas que faltaban' y 'ahora sí que funciona'.

Si mañana el jefe de estudios del IES El Caminàs nos llama porque la aplicación no arranca o introduce un error de cálculo, ¿alguien es capaz de saber en cuál de esos tres commits se rompió el código? Ninguno de nosotros. Hemos convertido un registro de ingeniería en una caja negra opaca.

*En una consultora profesional como AzaharTech, cada confirmación debe ser una **unidad atómica de cambio** perfectamente documentada. Para lograrlo, no podemos limitarnos a pulsar un botón sin entender qué ocurre por debajo: debemos dominar el viaje del archivo a través de los **tres estados de Git**, aprender a leer las **diferencias línea a línea (Git Diff)** y redactar mensajes bajo el estándar de la industria: **Conventional Commits**».*

---

2. El modelo de datos de Git: instantáneas frente a diferencias

La mayoría de los sistemas de control de versiones anteriores a Git (como CVS o Subversion) almacenaban la información como una lista de archivos y los cambios de texto aplicados a cada uno a lo largo del tiempo (diferencias basadas en deltas).

Git no funciona así. Git concibe la información como un conjunto de **instantáneas completas (*snapshots*)** de un sistema de archivos en miniatura:

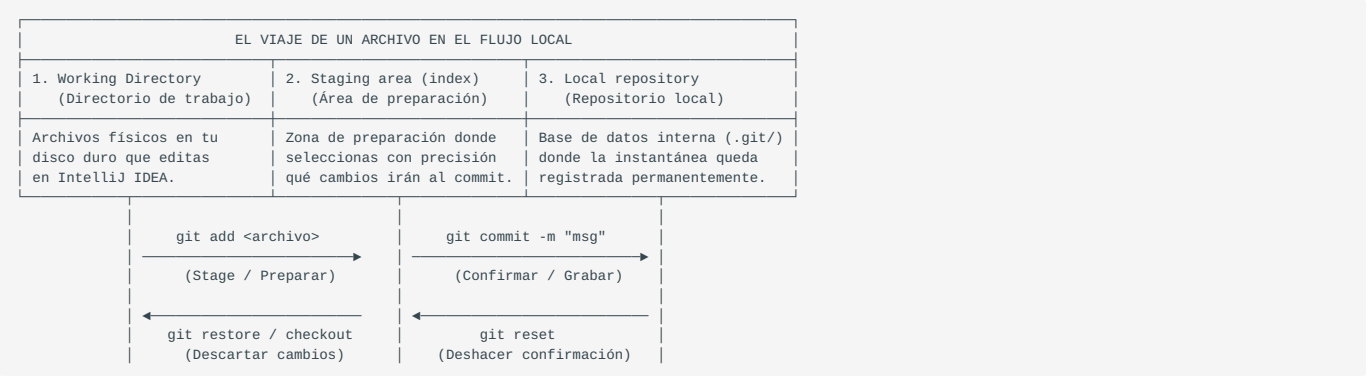
```
Versión 1: [ Archivo A ] [ Archivo B ] [ Archivo C ] ⟶ (Commit 1)
                                 |
                                 ▼ (Se modifica solo el archivo B)
Versión 2: [ Archivo A ] [ Archivo B' ] [ Archivo C ] ⟶ (Commit 2)
             (Enlace)                    (Enlace)
```

Cada vez que realizas un *commit*, Git «hace una fotografía» del estado exacto de todos los archivos del proyecto en ese momento y guarda una referencia a esa instantánea. Para ser eficiente, si un archivo no se ha modificado, Git no lo duplica: simplemente crea un enlace simbólico a la versión idéntica que ya tenía almacenada previamente.

---

3. Los tres estados locales de un archivo en Git

Para tener control absoluto sobre qué entra y qué no entra en cada fotografía del proyecto, Git divide tu espacio de trabajo local en **tres zonas lógicas**:



Estado 1. Modificado (*Modified*) — En el Working Directory

Has editado un archivo existente (añadiendo código en Java o cambiando un texto en Markdown), pero todavía no has confirmado esos cambios ni los has preparado para la foto. Los cambios solo existen en tu editor.

Estado 2. Preparado (*Staged*) — En el Staging Area

Has seleccionado voluntariamente un archivo modificado y le has dicho a Git: *«Quiero que esta modificación concreta forme parte de mi próxima instantánea»*. El comando de consola equivalente es `git add <nombre-archivo>`.

¿Por qué existe el Staging Area? Imagina que has trabajado durante una hora y has modificado dos cosas distintas: has corregido un cálculo matemático en `ControlAccesoQR.java` y has retocado el diseño del `README.md`. No debes mezclarlos en un único commit. El Staging Area te permite preparar primero `ControlAccesoQR.java`, hacer su commit específico de código, y después preparar `README.md` para hacer su commit de documentación.

Estado 3. Confirmado (*Committed*) — En el Local Repository

Los datos han quedado guardados de forma segura, comprimida y permanente en la base de datos interna de Git (la carpeta oculta `.git/` de tu proyecto). Cada commit genera un identificador alfanumérico único e irrepetible denominado **código hash SHA-1** (por ejemplo: `8f3a1b4c9e7...`), que actúa como la huella dactilar de esa versión.

---

4. Inspección de cambios: la lectura del *Git Diff*

Antes de preparar o confirmar un cambio, un ingeniero de software siempre inspecciona qué ha modificado mediante la herramienta **Git Diff** (comparador de diferencias).

El *diff* desglosa línea por línea las alteraciones exactas que ha sufrido el archivo: * **Líneas eliminadas o reemplazadas:** Se resaltan visualmente en **rojo** (o precedidas por el signo `-`). * **Líneas añadidas:** Se resaltan visualmente en **verde** (o precedidas por el signo `+`). * **Líneas en blanco / contexto:** Muestran el código circundante para ubicar exactamente en qué método o bloque se ha producido la intervención.

```
@@ -12,4 +12,5 @@ public class App {
     public static void main(String[] args) {
-        System.out.println("Sistema antiguo");
+        System.out.println("=== SISTEMA AZAHARTECH ===");
+        System.out.println("Terminal IES El Caminas v1.0");
     }
 }
```

Aprender a revisar el *diff* antes de hacer commit evita subir código experimental, comentarios personales o líneas que rompan la compilación del proyecto.

---

5. El estándar de calidad de la industria: *Conventional Commits*

Para evitar los mensajes ambiguos en los equipos de desarrollo, se utiliza la especificación **Conventional Commits v1.0.0**, un estándar internacional que añade significado estructurado a los mensajes de confirmación:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                        ESTRUCTURA DE UN CONVENTIONAL COMMIT                  │
├──────────────────────────────────────────────────────────────────────────────┤
│ <tipo>(<ámbito opcional>): <descripción concisa y directa en imperativo/presente>      │
└──────────────────────────────────────────────────────────────────────────────┘
```

CATÁLOGO DE TIPOS OFICIALES UTILIZADOS EN AZAHARTECH:

- `feat:` (de *feature*). Se utiliza exclusivamente cuando se introduce una nueva funcionalidad en el software (*ejemplo:* `feat(pr): calcular desglose horario en modulo de tiempos`).
- `fix:` Se emplea cuando se soluciona un error o defecto (*bug*) en el código fuente (*ejemplo:* `fix(pr): corregir perdida de decimales en ratio de asistencia`).
- `docs:` Modificaciones que afectan únicamente a la documentación técnica, diagramas o manuales Markdown (*ejemplo:* `docs(pi): documentar mapa de actores del proyecto`).
- `style:` Cambios visuales de formato que no alteran la lógica del programa: indentación, espacios en blanco, punto y coma omitidos (*ejemplo:* `style(pr): formatear espaciado segun guia oficial de Java`).
- `refactor:` Cambios en el código que ni corrigen un bug ni añaden una funcionalidad nueva, sino que mejoran su legibilidad o estructura (*ejemplo:* `refactor(pr): renombrar variables de tiempos a nomenclatura camelCase`).
- `chore:` Tareas auxiliares de configuración, mantenimiento de repositorios o actualización de herramientas que no modifican el código de producción (*ejemplo:* `chore(git): anadir exclusion de temporales .DS_Store a gitignore`).

LAS CUATRO REGLAS DE ORO DEL MENSAJE:

1. **Verbo en imperativo / presente:** Escribe "*añadir función*", no "*añadida*" ni "*añadiendo*".
2. **Minúsculas tras los dos puntos:** No empieces la descripción con mayúscula.
3. **Sin punto final:** Los mensajes de commit no llevan punto al final.
4. **Claridad atómica:** Un commit debe responder a la frase: *«Si aplico este commit, el proyecto ahora... [mensaje]»*.

Sesión 2. Laboratorio práctico guiado. Dominio del flujo Git en IntelliJ IDEA

Alba Torres y Pau Ferrer abren IntelliJ en sus pantallas:

«Vamos a realizar un entrenamiento práctico en vuestro entorno. Crearemos archivos de prueba, observaremos cómo cambian de estado en tiempo real dentro del IDE, inspeccionaremos las diferencias con la vista Diff de dos columnas y realizaremos commits atómicos con el estándar de Conventional Commits».

PASO 1. LOCALIZACIÓN DEL PANEL VISUAL DE GIT EN INTELLIJ

1. Abre tu proyecto en **IntelliJ IDEA**.
2. En la barra lateral izquierda, localiza y haz clic sobre el icono **Commit** (icono vertical con una marca de verificación) o pulsa el atajo de teclado universal: `Alt + 0` (en GNU/Linux).
3. Verás la ventana de control de versiones dividida en dos secciones principales:

- **Changes / Unstaged:** Archivos físicos que tienen cambios en tu disco duro, pero que aún no están listos para la instantánea.
- **Staged:** Archivos que ya forman parte del área de preparación.

PASO 2. MODIFICACIÓN DE ARCHIVOS Y OBSERVACIÓN DEL ESTADO *MODIFIED*

1. Despliega tu carpeta `ed/docs/` y crea un archivo de texto plano provisional llamado `bitacora.txt`.
2. Escribe dentro la siguiente línea:

```
Registro de pruebas de Git - Sesion 3
```

3. Guarda el archivo (`Ctrl + S`).
4. Observa el panel **Commit** de IntelliJ:

- El archivo aparece en color **rojo** (estado *Untracked* / sin seguimiento), lo que indica que Git ha detectado un archivo nuevo que no existía en el commit anterior.

PASO 3. INSPECCIÓN VISUAL CON EL VISOR DIFF DE DOS COLUMNAS

1. Haz doble clic sobre el archivo `bitacora.txt` en el panel de Commit (o selecciónalo y pulsa `Ctrl + D`).
2. Se abrirá la herramienta **Diff Viewer** de IntelliJ:

- A la izquierda verás la versión anterior del repositorio (vacía, porque el archivo no existía).
- A la derecha verás tu versión actual en disco, con la línea resaltada en color **verde**.

3. Cierra la pestaña de diferencias y abre el archivo `.gitignore` que creaste en la Semana 1.
4. Añade al final del archivo una nueva línea de comentario:

```
# Exclusiones de archivos temporales de registro
*.log
```

5. Abre el visor Diff sobre `.gitignore` : verás exactamente la línea que acabas de añadir en verde, mientras que el resto de reglas permanecen sin alteración.

PASO 4. PREPARACIÓN SELECTIVA (*STAGING*) Y DESCARTE DE CAMBIOS

1. En el panel de Commit de IntelliJ, selecciona únicamente el archivo `.gitignore` marcando su casilla de verificación.
2. Deja desmarcado el archivo `bitacora.txt` .
3. Esto equivale exactamente al comando de consola:

```
git add .gitignore
```

*El archivo `.gitignore` está ahora en el **Staging Area**, listo para su commit.*

PASO 5. REDACCIÓN DEL COMMIT CONVENCIONAL Y CONFIRMACIÓN LOCAL

1. En el cuadro de texto inferior del panel de Commit (*Commit Message*), escribe:

```
chore(git): añadir regla de exclusion para archivos de registro log
```

2. Observa cómo IntelliJ analiza el mensaje: debe ser claro y no superar los 72 caracteres de longitud.
3. Haz clic en el botón **Commit** (no en *Commit and Push* todavía).
4. El archivo `.gitignore` desaparece de la lista de cambios y queda registrado de forma permanente en tu base de datos local de Git.

PASO 6. DESCARTE DE CAMBIOS EN EL DIRECTORIO DE TRABAJO (*ROLLBACK*)

1. Supongamos que el archivo `bitacora.txt` era solo una prueba temporal que no queremos mantener en el proyecto.
2. En IntelliJ, haz clic derecho sobre `bitacora.txt` en el panel de cambios y selecciona **Rollback...** (o pulsa `Ctrl + Alt + Z`).
3. Confirma la acción: IntelliJ eliminará los cambios del archivo o lo borrará si era un archivo sin seguimiento, devolviendo tu directorio de trabajo a un estado 100 % limpio.

PASO 7. SINCRONIZACIÓN CON EL REPOSITORIO REMOTO (*PUSH*)

1. Ve al menú superior: **Git -> Push...** (o pulsa `Ctrl + Shift + K`).

2. Se abrirá un diálogo que te muestra la lista de commits locales pendientes de enviar a GitHub.
3. Comprueba que aparece tu commit con el mensaje `chore(git): añadir regla de exclusion...`.
4. Haz clic en el botón **Push**.
5. Abre tu navegador web, entra en tu repositorio de GitHub y comprueba que en el historial (*Commits*) aparece tu nueva confirmación impecablemente registrada.

2.2.2 Día 4 - 1 sesión

Teoría. El estándar markdown en la ingeniería del software

1. Caso guía AzaharTech

Es viernes por la tarde en la sala de desarrollo de **AzaharTech**. **Alba Torres** proyecta en la pantalla dos documentos técnicos: uno es un archivo `.docx` de Microsoft Word y el otro es un archivo `.md` de Markdown renderizado en GitHub:

*«Fijaos en la diferencia. En el archivo Word, para ver qué ha cambiado entre dos versiones hay que descargarlo, tener el programa de pago instalado y aceptar revisiones manuales. En cambio, el archivo **Markdown (`.md`)** es texto plano universal. Git puede ver cada coma modificada línea por línea y GitHub lo maqueta con tipografía profesional automáticamente.*

*En los proyectos de AzaharTech no aceptamos documentación viva en formatos binarios propietarios. Hoy aprenderemos la sintaxis de Markdown, redactaremos la **memoria técnica de nuestro marco Scrum (Entregable ED-2)** y transformaremos el archivo `README.md` en el panel de control del proyecto».*

---

2. Sintaxis esencial de Markdown para documentación de software

Markdown fue creado para ser legible en su forma de texto plano sin procesar y convertirse limpiamente en documentos maquetados.

````
┌──────────────────────────────────────────────────────────────────────────────┐
│                        GUÍA DE SINTAXIS ESENCIAL DE MARKDOWN                 │
├──────────────────────────────┬───────────────────────────────────────────────┤
│ Elemento                     │ Sintaxis en texto plano                       │
├──────────────────────────────┼───────────────────────────────────────────────┤
│ Encabezado principal (H1)    │ # Título del Proyecto                         │
│ Encabezado secundario (H2)   │ ## 1. Marco Metodológico                      │
│ Subapartado (H3)             │ ### 1.1 Fases del Ciclo de Vida               │
│ Texto en negrita             │ **Texto en negrita para conceptos clave**     │
│ Texto en cursiva             │ *Texto en cursiva o términos en inglés*       │
│ Lista no ordenada            │ * Elemento de lista (o guion -)               │
│ Lista de tareas (Checklist)  │ - [ ] Tarea pendiente / - [x] Tarea completada │
│ Bloque de código con sintaxis │ ```` ```java ```` seguido del código y ```` ``` ````  │
│ Cita o nota importante       │ > Nota técnica o advertencia                  │
└──────────────────────────────┴───────────────────────────────────────────────┘
````

TABLAS EN MARKDOWN:

Las tablas se construyen mediante barras verticales (`|`) y una línea divisoria de guiones (`---`):

```
| Identificador | Tarea del Sprint | Módulo Responsable | Estado |
| :--- | :--- | :---: | :---: |
| T-01 | Configuración OpenJDK 21 | ED | Completada |
| T-02 | Proyecto en Java | PR | En curso |
```

(Los dos puntos `:` determinan la alineación: `:---` izquierda, `:---:` centrada, `---:` derecha).

Laboratorio práctico guiado. Redacción de ED-2 y panel README

PASO 1. REDACCIÓN DE LA MEMORIA TÉCNICA SCRUM: `ed/docs/marco-scrum.md` (ENTREGABLE ED-2)

1. En tu proyecto de IntelliJ, navega hasta la carpeta `ed/docs/`.
2. Haz clic derecho sobre `docs` : **New -> File** y nómbralo exactamente: `marco-scrum.md` .

3. Redacta el documento técnico formal aplicando la sintaxis Markdown y adaptándolo a **tu proyecto elegido de la bolsa de proyectos**:

```
# Memoria Técnica: Marco Metodológico y Ciclo de Vida del Software
**Consultora:** AzaharTech Software Consulting
**Proyecto:** [nombre de tu proyecto elegido de la bolsa de proyectos]
**Desarrollador/a:** [tus apellidos, tu nombre]
**Fecha:** 25 de septiembre de 2026
**Versión:** 1.0 (Sprint 1)

---

## 1. Justificación del modelo de proceso: Cascada vs. Scrum
Para el desarrollo de este proyecto se descarta el modelo tradicional en cascada debido a su rigidez ante los cambios de requisitos y a la dilatación en la entrega de resultados tangibles al cliente.

Se adopta el marco de trabajo ágil **Scrum**, estructurado en ciclos iterativos de desarrollo (**Sprints de 3 semanas**). Al finalizar cada sprint, el equipo entrega un **incremento de software potencialmente desplegable**, permitiendo que el cliente valide el producto de forma continua y minimizando el riesgo de desviación temporal o económica.

---

## 2. Aplicación de las fases del Ciclo de Vida del Software (SDLC)
En cada ciclo de sprint se ejecutan de forma coordinada las fases de la ingeniería del software adaptadas a nuestro sistema:

1. **Análisis.** Identificación de las necesidades del cliente y especificación de historias de usuario con criterios de aceptación claros.
2. **Diseño.** Modelado de la arquitectura de la solución, diagramas funcionales en Proyecto Intermodular y definición de estructuras de datos.
3. **Codificación.** Implementación de los algoritmos en lenguaje Java (OpenJDK 21) utilizando el entorno integrado IntelliJ IDEA Community.
4. **Pruebas (Testing).** Verificación de casos límite, validación de trazas de memoria e inspección visual con depurador.
5. **Despliegue.** Empaquetado y publicación de las entregas formales mediante etiquetas de versión (*Git Tags*) en el repositorio de GitHub.
6. **Mantenimiento.** Refactorización continua del código y corrección de incidencias detectadas en revisiones de sprint.

---

## 3. Organización y Roles en el Equipo de Trabajo
* **Product Owner.** Representa los intereses del cliente de nuestro proyecto, priorizando los requisitos en el *Product Backlog*.
* **Scrum Master (Laia Claramunt).** Supervisa el cumplimiento de los tiempos de entrega, elimina bloqueos técnicos y vela por la calidad metodológica.
* **Developer (El estudiante).** Responsable técnico del diseño algorítmico, implementación en Java, control de versiones en Git y documentación técnica.
```

PASO 2. TRANSFORMACIÓN DEL `README.md` EN PANEL DE CONTROL DEL PROYECTO

1. Abre el archivo `README.md` situado en la raíz de tu carpeta personal (`azahartech/nombre-equipo/apellidos-nombre/README.md`).
2. Reemplaza su contenido para que actúe como la tarjeta de presentación oficial de tu espacio de trabajo:

```
# Sistema de Gestión: [nombre de tu proyecto propio]
**Consultora de Desarrollo:** AzaharTech (Castellón de la Plana)
**Cliente:** [nombre del cliente de la bolsa de proyectos]
**Desarrollador/a:** [tus apellidos, tu nombre]
**Equipo:** [nombre de tu equipo asignado]

---

## Alcance del proyecto
Desarrollo de una solución informática multiplataforma para optimizar los procesos operativos y de control de nuestro cliente, integrando lógica de procesamiento en Java, gestión ágil de tareas y documentación técnica viva.

---

## Taller tecnológico
* **Lenguaje:** Java (OpenJDK 21 LTS)
* **Entorno Integrado (IDE):** IntelliJ IDEA Community Edition
* **Control de Versiones:** Git 2.x & GitHub
* **Estándar Documental:** Markdown

---

## Sprint Backlog 1 (14 sep - 2 oct) — Estado de avance

### Módulo: Entornos de Desarrollo (ED)
- [x] Configuración de OpenJDK 21 e IntelliJ IDEA Community (`ed/docs/entorno.png`)
- [x] Elaboración de memoria técnica de marco Scrum y ciclo de vida (`ed/docs/marco-scrum.md`)
- [x] Práctica del flujo de Git de 3 estados y estándar Conventional Commits
- [ ] Auditoría de limpieza e higiene con `.gitignore` (Semana 3)
- [ ] Etiquetado formal de release `v0.1.0-sprint1` (Semana 3)

### Módulo: Programación (PR)
- [x] Declaración de variables y tipos primitivos de datos
- [x] Operadores aritméticos, expresiones y conversiones de tipo (*casting*)
- [ ] Algoritmo secuencial completo del proyecto (Semana 3)

### Módulo: Proyecto Intermodular (PI)
- [x] Documento de especificación del proyecto y registro ético de IA (`pi/docs/analisis-proyecto.md`)
- [x] Diagrama de bloques funcional y viabilidad técnica (`pi/docs/viabilidad-tecnica.md`)
- [ ] Dossier técnico final y preparación de la Demo v0.1 (Semana 3)
```

PASO 3. CONFIRMACIÓN ATÓMICA Y SINCRONIZACIÓN CON GITHUB

1. Abre el panel **Commit** en IntelliJ IDEA (`Alt + 0`).
2. Selecciona los dos archivos modificados: `ed/docs/marco-scrum.md` y `README.md` .
3. Escribe un mensaje de confirmación convencional riguroso:

```
docs(ed): redactar memoria tecnica de marco scrum y actualizar panel README
```

4. Haz clic en **Commit and Push**.
5. Accede a tu repositorio en GitHub desde el navegador:

- Comprueba que el `README.md` se muestra maquetado en la portada con sus casillas de verificación activas.
- Entra en `ed/docs/marco-scrum.md` y comprueba que la memoria de Scrum se visualiza con títulos limpios, tablas y negritas.

2.3 Semana 3. Cierre de sprint e higiene (.gitignore)

2.3.1 Día 5 - 2 sesiones

Sesión 1. Teoría. El cierre del Sprint y la calidad del repositorio

1. Caso guía en AzaharTech. La recta final del Sprint 1

Es lunes 28 de septiembre por la tarde. Entramos en la última semana del primer ciclo de desarrollo. En la sala de reuniones de **AzaharTech**, **Pau Ferrer** y **Alba Torres** revisan el código de sus respectivos módulos con cierta euforia: el programa compila, las fórmulas matemáticas funcionan y los archivos están redactados en Markdown.

Pau comenta entusiasmado:

«Por mi parte el Sprint 1 está terminado: ya he subido todo lo que tenía en mi ordenador a GitHub».

Laia Claramunt, con su experiencia como Scrum Master, sonríe y niega con la cabeza:

«En el desarrollo de software profesional, un sprint no concluye cuando el programador termina de escribir la última línea de código. Eso es solo la mitad del trabajo.

*En Scrum, el final de un sprint exige ejecutar dos ceremonias obligatorias: la **Sprint Review**, donde demostramos al cliente el software que funciona y recibimos su validación, y la **Sprint Retrospective**, donde nos miramos al espejo como equipo para detectar qué fallos de organización hemos tenido y cómo los resolveremos en el Sprint 2.*

*Además, antes de presentar nada al cliente, debemos realizar una **auditoría de higiene técnica sobre nuestro repositorio**. Si abro vuestro GitHub y encuentro archivos compilados `.class`, carpetas temporales de IntelliJ o rutas rotas, el trabajo no se considerará entregado. Un desarrollador junior solo se fija en si el código compila; un profesional se asegura de que el repositorio esté impecable».*

---

2. Las ceremonias de cierre del Sprint: Review frente a Retrospective

En los modelos predictivos clásicos (*modelo en cascada*), los proyectos terminaban con una entrega masiva y fría de documentos al cabo de meses. Scrum introduce dos eventos de inspección y adaptación al término de cada ciclo de 3 semanas:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      COMPARATIVA DE LAS CEREMONIAS DE CIERRE             │
├──────────────────────────┬───────────────────────────────────────────────┤
│ SPRINT REVIEW (revisión) │ SPRINT RETROSPECTIVE (retrospectiva)          │
├──────────────────────────┼───────────────────────────────────────────────┤
│ • Participan: equipo + cliente │ • Participan: únicamente el equipo de desarrollo │
│ • Pregunta: ¿QUÉ se ha hecho? │ • Pregunta: ¿CÓMO hemos trabajado juntos?  │
│ • Foco: el producto / software │ • Foco: los procesos, las personas y las herramientas │
```

```
| • Salida: feedback del cliente| • Salida: compromisos de mejora interna para el Sprint 2|
└──────────────────────────────┴──────────────────────────────────────────────────────────┘
```

A. LA REVISIÓN DEL SPRINT (*SPRINT REVIEW*)

- **Objetivo** Inspeccionar el **Incremento de Software** completado durante las 3 semanas y adaptar el *Product Backlog* según las impresiones del cliente.
- **Dinámica.** El equipo realiza una demostración en vivo del software funcionando (la clase `MiProyecto.java` del caso guía o de su proyecto propio). Se explican las decisiones tomadas y se verifica si se ha cumplido la **Definición de Hecho (*Definition of Done - DoD*)**.
- **El valor del feedback.** El cliente puede solicitar ajustes para el siguiente sprint (*ejemplo: «necesitamos que el tiempo de espera no supere los 5 segundos»*), los cuales se incorporan de forma natural al backlog del Sprint 2.

B. LA RETROSPECTIVA DEL SPRINT (*SPRINT RETROSPECTIVE*)

- **Objetivo.** Analizar la salud operativa del equipo de desarrollo, identificando qué prácticas han funcionado y qué obstáculos técnicos o humanos han frenado el ritmo de trabajo.
- **El método de las 3 columnas de AzaharTech:**

a. **Mantener (*Keep*).** Buenas prácticas consolidadas (*por ejemplo, diseñar en PSeInt antes de picar Java ahorraron errores; los commits convencionales facilitaron rastrear cambios*).

b. **Detener (*Stop*).** Prácticas perjudiciales que deben erradicarse (*por ejemplo, dejar la redacción de la memoria Markdown para la última noche; hacer commits gigantes con 15 archivos a la vez*).

c. **Comenzar (*Start*).** Nuevos hábitos o herramientas a incorporar en el Sprint 2 (*por ejemplo, revisar los mensajes de commit con los compañeros antes de hacer push; configurar atajos de teclado en el IDE*).

---

3. Principios de higiene técnica. El control estricto de `.gitignore`

Uno de los errores más graves que comete un desarrollador novel al utilizar Git es confundir un **repositorio de código fuente** con una **carpeta compartida en la nube** (como Google Drive o NextCloud).

¿POR QUÉ JAMÁS DEBEN SUBIRSE ARCHIVOS BINARIOS O TEMPORALES AL REPOSITORIO?

1. **Contaminación del historial.** Los archivos compilados (`.class` , `.jar` , `.exe`) son binarios que cambian por completo cada vez que recompilas. Si los subes a Git, la base de datos del repositorio aumentará de tamaño de forma exponencial con información inútil.
2. **Conflictos entre sistemas operativos.** Si subes la carpeta oculta `.idea/` o los archivos de configuración local de tu ordenador con Windows, cuando otro compañero (o el profesor que califica en GNU/Linux) clone el repositorio, el proyecto colapsará porque las rutas de disco no coincidirán.
3. **Seguridad.** Los archivos temporales pueden contener credenciales de sesión, rutas absolutas privadas de tu disco duro o cachés del sistema.

```
[ REPOSITORIO PROFESIONAL LIMPIO ]            [ REPOSITORIO CONTAMINADO ]
├── ed/docs/marco-scrum.md   (Texto editable)   ├── ed/docs/marco-scrum.md
├── pr/src/App.java          (Código fuente)    ├── pr/src/App.java
└── .gitignore               (Reglas activas)   ├── pr/src/App.class   ◄── [ERROR GRAVE: binario]
                                                ├── out/production/... ◄── [ERROR GRAVE: salida]
                                                └── .idea/workspace.xml◄── [ERROR GRAVE: privado]
```

EL ARCHIVO `.gitignore` : EL GUARDIÁN DE LA LIMPIEZA

El archivo `.gitignore` situado en la raíz del proyecto le indica al motor de Git qué archivos y directorios debe **ignorar por completo**, impidiendo que aparezcan en el *Staging Area* aunque se modifiquen en el disco duro.

- **Patrones y comodines estándar:**
- `*.class` : Ignora cualquier archivo que termine con esa extensión, en cualquier subcarpeta.
- `out/` : Ignora la carpeta de compilación de IntelliJ y todo su contenido.
- `.idea/` : Ignora las configuraciones privadas del IDE.
- `.DS_Store` : Ignora los archivos de previsualización ocultos de macOS.

Sesión 2. Laboratorio práctico guiado. Auditoría de higiene y purga del repositorio

Alba Torres y Pau Ferrer abren la terminal integrada de IntelliJ:

«Ha llegado el momento de auditar nuestro repositorio antes de la entrega formal. Vamos a comprobar el estado de los archivos con Git, aprenderemos a eliminar del seguimiento cualquier archivo temporal que se haya colado por accidente y consolidaremos el archivo `.gitignore` definitivo de AzaharTech».

Paso 1. Auditoría visual de archivos en el árbol de proyectos de IntelliJ

1. Abre tu proyecto en **IntelliJ IDEA**.
2. Despliega la ventana lateral del explorador de proyectos pulsando el atajo `Alt + 1`.
3. Observa el código de colores que utiliza IntelliJ para indicar el estado de cada archivo respecto a Git:

- **Color gris o amarillo apagado.** Archivo o carpeta ignorada por la configuración de Git (estado correcto para archivos temporales).
- **Color blanco o texto normal.:** Archivo versionado y confirmado en el repositorio local.
- **Color azul.** Archivo existente modificado que aún no se ha confirmado.
- **Color rojo.** Archivo nuevo detectado en el disco que todavía no está bajo control de versiones.

4. **Revisión de carpetas clave:**

- Despliega la carpeta `pr/src/`: solo deben aparecer archivos con extensión `.java`. No debe verse ningún archivo de color blanco que termine en `.class`.
- Observa las carpetas `.idea/` y `out/`: deben aparecer en color grisáceo o atenuado.
- **Alerta visual:** si las carpetas `.idea/`, `out/` o algún archivo `.class` aparecen con el nombre en texto blanco normal, significa que Git los está rastreando por error y deben retirarse del control de versiones.

Paso 2. Retirada visual de archivos no deseados de Git (sin borrarlos del disco)

Si en las semanas anteriores se confirmó por error una carpeta temporal o un archivo compilado, se puede corregir en tres pasos:

1. **Borrar los archivos en IntelliJ.** Selecciona la carpeta `out/` o los archivos `.class` en el explorador de proyectos (`Alt + 1`) y pulsa la tecla `Supr` (Delete).
2. **Confirmar y subir la eliminación.** Abre el panel de Commit (`Ctrl + K`), escribe el mensaje `chore(git): eliminar archivos compilados del repositorio` y pulsa **Commit and Push**.
3. **Reconstruir el proyecto.** Ve al menú superior **Build -> Rebuild Project**. IntelliJ volverá a generar los archivos en tu disco duro para que el programa funcione, pero como la regla ya está en el `.gitignore`, ahora aparecerán en color grisáceo y Git no los volverá a rastrear.

PASO 3. CONSOLIDACIÓN DE LAS REGLAS UNIVERSALES EN `.gitignore`

1. Abre el archivo `.gitignore` situado en la raíz de tu proyecto haciendo doble clic sobre él en IntelliJ IDEA.
2. Comprueba que contiene las directivas de exclusión estructuradas por bloques:

```
# ====================================================================
# exclusiones oficiales de azahartech software
# ====================================================================

# 1. entorno de desarrollo (IntelliJ IDEA)
.idea/
*.iml
*.iws
*.ipr
out/

*.class
```

```
target/
build/
*.jar
*.war

# 3. archivos del sistema operativo
.DS_Store
Thumbs.db
desktop.ini

# 4. archivos de registro y temporales
*.log
*.tmp
*.bak
```

1. Guarda los cambios con el atajo `Ctrl + S`.
2. Observa cómo, de forma instantánea, todas las carpetas y extensiones indicadas cambian automáticamente a color gris atenuado en el árbol del proyecto.

PASO 4. CONFIRMACIÓN Y SINCRONIZACIÓN VISUAL CON GITHUB

1. Abre el panel lateral **Commit** de IntelliJ mediante el atajo `Ctrl + K` (o pulsa en el icono verde de verificación en la barra lateral izquierda).
2. En la lista de cambios preparados (*Staged* o *Changes*), comprueba que aparecen seleccionados:

- La modificación del archivo `.gitignore`.
- Las órdenes de retirada de archivos temporales si tuviste que aplicar el paso 2.

3. En la caja de texto inferior, redacta el mensaje de confirmación siguiendo el estándar convencional:

```
chore(git): auditar higiene del repositorio y consolidar reglas de gitignore
```

4. Despliega el botón azul inferior y pulsa **Commit and Push**.
5. En la ventana emergente de confirmación, revisa el envío y pulsa **Push**.
6. Abre tu navegador web, entra en tu repositorio de GitHub y comprueba:

- Que en la raíz no existe ninguna carpeta `.idea/` ni `out/`.
- Que dentro de `pr/src/` únicamente existen los archivos fuente con extensión `.java`.

2.3.2 Día 6 - 1 sesión

Teoría. Versionado semántico y etiquetas (*TAGS*)

1. Caso guía AzaharTech

Es viernes 2 de octubre por la tarde. Las tres semanas del Sprint 1 concluyen hoy formalmente. **Laia Claramunt** reúne por última vez en este sprint a todo el equipo de desarrollo de AzaharTech:

«Equipo, el lunes que viene comenzaremos el Sprint 2: modificaremos clases en Programación, cambiaremos la configuración de Maven en Entornos de Desarrollo y ampliaremos la documentación.

Si el docente o el cliente entran dentro de dos semanas a nuestro repositorio para evaluar el Sprint 1, ¿cómo sabrán exactamente qué archivos habíamos terminado hoy viernes 2 de octubre a las doce y cuáles pertenecen al trabajo posterior del Sprint 2?

*No podemos fiarnos de la memoria. En ingeniería del software los hitos no se guardan duplicando carpetas con nombres como 'Sprint1_Final_Bueno'. Se sellan en el historial mediante **etiquetas de versión (Git Tags)** bajo el estándar de **Versionado Semántico**. Hoy congelaremos nuestro primer incremento de software con la etiqueta oficial `v0.1.0-sprint1`».*

---

2. Fundamentos del versionado semántico y las etiquetas en Git

```
commit a1b2 (feat) ──► commit c3d4 (docs) ──► commit e5f6 (chore) ──► commit 7a8b (docs)
                                                                          |
                                                                  [ TAG: v0.1.0-sprint1 ]
                                                                  (Puntero estático e inmutable)
```

A. ¿QUÉ ES UN GIT TAG Y EN QUÉ SE DIFERENCIA DE UNA RAMA?

- **Etiqueta (*Tag*).** Es una marca fija, estática e inmutable que apunta a un commit específico del historial. Una vez creada, **jamás se mueve**. Aunque el proyecto continúe recibiendo cientos de commits en el futuro, el tag siempre permitirá recuperar el estado exacto del código en el instante preciso en que se creó.
- **Rama (*Branch*).** Es un puntero móvil que avanza automáticamente cada vez que realizas un nuevo commit. Se utiliza para desarrollar tareas activas.

B. EL ESTÁNDAR DE VERSIONADO SEMÁNTICO (*SEMVER 2.0.0*)

El estándar de la industria establece que las versiones del software deben seguir el formato numérico:

```
v X . Y . Z
  |   |   |
  |   |   └──► PATCH (parche). Corrección de fallos menores manteniendo compatibilidad.
  |   └──────► MINOR (menor). Nuevas funcionalidades que no rompen lo anterior.
  └──────────► MAJOR (mayor). Cambios estructurales profundos incompatibles con lo previo.
```

- **Para nuestro Sprint 1.** Al tratarse del primer prototipo inicial del curso, adoptamos la versión base pre-lanzamiento: `v0.1.0-sprint1`.

Laboratorio práctico guiado. Firma de la Release y entrega oficial

PASO 1. ACTUALIZACIÓN FINAL Y CIERRE DEL SPRINT BACKLOG 1 EN `README.md`

1. Abre tu proyecto en IntelliJ IDEA.
2. Abre el archivo `README.md` situado en la raíz de tu carpeta personal.
3. Edita la sección del backlog para marcar **todas las casillas al 100 % de cumplimiento** sustituyendo los espacios `[ ]` por una `x` minúscula `[x]`:

```
## 📋 Sprint Backlog 1 (14 sep - 2 oct) — Estado final: 100% COMPLETADO

### Módulo: Entornos de Desarrollo (ED)
- [x] Configuración de OpenJDK 21 e IntelliJ IDEA Community (`ed/docs/entorno.png`)
- [x] Elaboración de memoria técnica de marco Scrum y ciclo de vida (`ed/docs/marco-scrum.md`)
- [x] Práctica del flujo de Git de 3 estados y estándar Conventional Commits
- [x] Auditoría de limpieza e higiene técnica con `.gitignore` consolidado
- [x] Etiquetado formal de release `v0.1.0-sprint1` publicado en GitHub

### Módulo: Programación (PR)
- [x] Declaración de variables y tipos primitivos de datos (`pr/src/`)
- [x] Operadores aritméticos, expresiones y conversiones de tipo (*casting*)
- [x] Programa secuencial completo del Reto 1 integrado con `printf`

### Módulo: Proyecto Intermodular (PI)
- [x] Documento de especificación del reto y registro ético de IA (`pi/docs/analisis-reto.md`)
- [x] Diagrama de bloques funcional y viabilidad técnica (`pi/docs/viabilidad-tecnica.md`)
- [x] Dossier técnico final y preparación de la Demo v0.1
```

1. Realiza el último commit de documentación del sprint desde el panel de Commit de IntelliJ:

```
docs: cerrar sprint backlog 1 al 100% de cumplimiento
```

2. Haz **Push** a GitHub.

PASO 2. CREACIÓN DE LA ETIQUETA DE VERSIÓN FORMAL (*GIT TAG*)

1. En el menú superior de IntelliJ, ve a: **Git -> New Tag...**
2. Se abrirá una ventana emergente:
3. **Tag Name.** Escribe exactamente `v0.1.0-sprint1` (todo en minúsculas y sin espacios).
4. **Commit.** Déjalo por defecto en `HEAD` (apunta al último commit que acabas de hacer).

5. **Message.** Escribe la descripción de la entrega:

```
Release oficial del Sprint 1 - Incremento Base AzaharTech
```

6. Haz clic en el botón **Create Tag**.

PASO 3. PUBLICACIÓN DEL TAG EN EL SERVIDOR REMOTO DE GITHUB (*PUSH TAGS*)

Por defecto, cuando realizas un `git push`, Git solo envía los nuevos commits de las ramas, **pero no sube las etiquetas**. Para sincronizar los tags con GitHub:

1. **Desde la interfaz de IntelliJ:**
2. Ve al menú superior: **Git -> Push...** (o pulsa `Ctrl + Shift + K`).
3. En la esquina inferior izquierda de la ventana emergente, marca obligatoriamente la casilla de verificación: **Push Tags: All** (o selecciona el tag `v0.1.0-sprint1`).
4. Haz clic en **Push**.

PASO 4. VERIFICACIÓN FINAL EN LA INTERFAZ WEB DE GITHUB (LISTA DE COTEJO DE LA ENTREGA)

1. Abre tu navegador web y entra en la página principal de tu repositorio en GitHub.
2. Realiza las siguientes cuatro comprobaciones visuales de calidad:
3. **Portada (`README.md`).** Verifica que todas las tareas del Sprint Backlog 1 aparecen con su casilla marcada (`[x]`).
4. **Sección Tags / Releases.** En la columna derecha de la pantalla de inicio de GitHub, comprueba que aparece la sección **Tags** y que al pulsar sobre ella se visualiza la etiqueta **`v0.1.0-sprint1`** con su fecha de creación.
5. **Higiene.** Comprueba que en el listado de archivos no existen carpetas `.idea/`, ni `.class`, ni `out/`.
6. **Estructura oficial.** Entra en `ed/docs/` y verifica que existen `entorno.png` (Entregable ED-1) y `marco-scrum.md` (Entregable ED-2).

Resumen del Sprint 1 de Entornos de Desarrollo completado

En estas 6 sesiones (9 horas lectivas en total): 1. Has interiorizado los fundamentos de la **ingeniería del software (SDLC)** y el marco ágil **Scrum**. 2. Cuentas con un taller de desarrollo profesional verificado (**OpenJDK 21 + IntelliJ IDEA**). 3. Dominas el ciclo de vida de **Git** (los 3 estados, *Git Diff*, *Conventional Commits* y *Git Tags*). 4. Tu repositorio en **GitHub** está impecablemente estructurado, documentado con **Markdown** y congelado bajo la versión oficial **`v0.1.0-sprint1`**, 100 % listo para ser calificado con la lista de cotejo de Entornos de Desarrollo.

3. Unidad 2

~~# Sprint 2. El viaje del código: compilación, máquinas virtuales y dependencias

3.1 Semana 4. Código fuente, bytecode y JVM

3.1.1 Día 7 - 2 sesiones

Primera sesión: teoría. La transformación del software: de las palabras humanas al bytecode universal

1. Caso guía en AzaharTech

Son las tres de la tarde del lunes 5 de octubre de 2026. En la sede de **AzaharTech** en Castellón de la Plana, el equipo de desarrollo inicia la primera jornada del **Sprint 2**.

Tras haber cerrado el primer sprint con las bases algorítmicas secuenciales y el taller digital con Git, **Laia Claramunt** conecta su equipo al proyector de la sala de reuniones. En la pantalla aparece un correo del responsable técnico del **IES El Caminàs**:

«Equipo, entramos en una nueva etapa del proyecto. El coordinador de informática del IES El Caminàs nos plantea una duda fundamental: en el centro educativo tienen servidores con Linux, ordenadores de gestión en conserjería con Windows y terminales portátiles variados. Nos preguntan si tendremos que programar tres versiones distintas de la aplicación para que funcione en cada uno de sus sistemas.

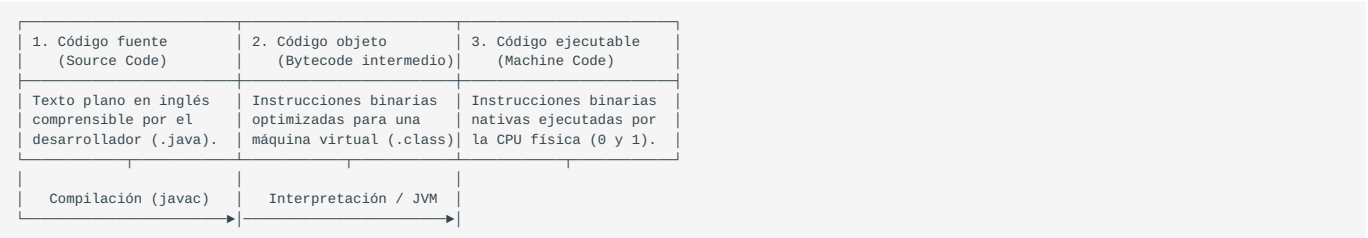
*La respuesta es un no rotundo, y la razón técnica reside en **cómo viaja y se transforma el código en Java**.*

*En los proyectos que vuestros equipos están desarrollando de la bolsa de proyectos ocurre lo mismo: tanto si estáis con la **Avent ura conversacional**, el **Motor de recomendación**, el **Simulador de físicas 2D** o la **Bóveda de contraseñas**, el software debe ser independiente de la máquina física.*

*Hoy aprenderemos a diferenciar con precisión el **código fuente**, el **código objeto (bytecode)** y el **código máquina ejecutable**. Descubriremos qué hace realmente el compilador cuando pulsamos un botón y miraremos por primera vez dentro de un archivo compilado».*

2. La cadena de transformación del código

Para que una orden escrita en lenguaje humano llegue a mover los circuitos electrónicos de un procesador, el software atraviesa tres estados claramente diferenciados:



A. CÓDIGO FUENTE (*SOURCE CODE*)

Es el archivo de texto editable que escribe el programador en el IDE (`ControlAccesoQR.java` o `MiProyecto.java`). Contiene palabras reservadas (`public` , `class` , `int` , `double`), identificadores y comentarios. La CPU no puede ejecutar este archivo directamente porque no entiende texto plano.

B. CÓDIGO OBJETO INTERMEDIO (*BYTECODE*)

Es el resultado de someter el código fuente al proceso de compilación mediante la herramienta `javac`. * No es código máquina específico de Intel, AMD o ARM. * Es un conjunto de instrucciones binarias compactas diseñadas para un procesador abstracto e ideal: la **Máquina Virtual de Java**. * Se almacena en archivos con extensión `.class`.

C. CÓDIGO EJECUTABLE (*MACHINE CODE*)

Es el código binario final formado por ceros y unos (`0` y `1`) que la arquitectura física de una CPU concreta (x86_64, ARM) sabe descodificar en sus registros electrónicos.

---

3. Tecnologías de virtualización: ¿por qué Java no compila a binario directo?

Los lenguajes tradicionales como C o C++ compilan el código fuente directamente a código máquina nativo del sistema operativo donde se compila.

```
MODELO TRADICIONAL (C / C++):
[ Código C ] ──(Compilador Windows)──► [ ejecutable.exe (Solo funciona en Windows) ]
[ Código C ] ──(Compilador Linux)────► [ binario_elf   (Solo funciona en Linux)   ]

MODELO DE VIRTUALIZACIÓN (Java):
[ Código Java (.java) ] ──(javac)──► [ Bytecode (.class) ] ──(Universal)
                                                │
                                ┌───────────────┼───────────────┐
                                ▼               ▼               ▼
                        [ JVM para Windows ]  [ JVM para Linux ]  [ JVM para macOS ]
```

- **El problema del modelo tradicional.** Para que el IES El Caminàs pudiera usar el software en distintos equipos, habría que mantener compiladores distintos y adaptar llamadas al sistema operativo en cada plataforma.
- **La solución de la virtualización (JVM).** Java introduce la **Máquina Virtual de Java (Java Virtual Machine - JVM)** como capa intermedia de software:

a. El desarrollador compila una sola vez y genera el archivo `.class`.

b. Ese mismo archivo `.class` viaja a cualquier ordenador del mundo.

c. La JVM instalada en el ordenador del cliente lee el Bytecode y lo traduce en tiempo real a las instrucciones de su CPU local.

d. Así se cumple el principio fundamental: **«Write Once, Run Anywhere»** (Escribe una vez, ejecuta en cualquier parte).

---

4. Laboratorio práctico guiado. Inspección de las tripas de un archivo `.class`

Vas a comprobar con tus propios ojos la diferencia física entre código fuente y Bytecode:

1. En el explorador de archivos o en IntelliJ IDEA, localizad la carpeta donde el entorno guarda los archivos compilados: `out/production/Proyecto/`
2. Abrid con un visor de texto plano el archivo **`ControlAccesoQR.class`**:

- ¿Qué se visualiza? Veréis caracteres extraños, símbolos binarios y, en las cuatro primeras posiciones hexadecimales, la firma mágica histórica con la que comienzan todos los archivos de Java en el mundo: **`0xCAFEBABE`**.

3. **Uso del desensamblador del JDK (`javap`):**

- El JDK incluye una herramienta oficial para traducir el Bytecode a mnemónicos legibles por los informaticos.
- Abrid la terminal integrada de IntelliJ (`Alt + F12`) y ejecutad:

```
javap -c out/production/*/ControlAccesoQR.class
```

- Observad la salida en pantalla: veréis las instrucciones internas reales de la máquina virtual de Java que generó vuestro código (mnemónicos como `iload`, `bipush`, `imul`, `invokevirtual`).
- *Pregunta para el grupo:* ¿Identificáis las operaciones de multiplicación (`imul`) y el número 60 (`bipush 60`) que programamos la semana pasada para el cálculo de estancia? El código que escribimos en Java se ha convertido en esas órdenes exactas.

Sesion 2. Dojo de entrenamiento y katas de herramientas

Pasamos al dojo de entrenamiento de herramientas, con tres **katas de compilación, ejecución y análisis de máquina virtual** para entrenar la independencia del software respecto al entorno de desarrollo, el desacoplamiento de binarios y la inspección de bajo nivel de la JVM:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). COMPILACIÓN Y EJECUCIÓN MULTI-ENTORNO DE TU PROYECTO PROPIO

- **Objetivo.** Demostrar empíricamente que el código fuente de tu proyecto no depende de IntelliJ IDEA y es capaz de compilarse y ejecutarse limpiamente desde la consola del sistema operativo utilizando únicamente las herramientas del JDK (`javac` y `java`).
- **Aplicación según tu proyecto elegido:**
- En **Aventura conversacional**. Compilar manualmente `MiProyecto.java` desde la terminal en `pr/src/` , verificar la generación del archivo `MiProyecto.class` , ejecutarlo comprobando la captura de datos del personaje y limpiar después el archivo binario residual generado en `src/` .
- En **Motor de recomendación**. Compilar manualmente `MiProyecto.java` desde la terminal en `pr/src/` , verificar que la JVM ejecuta el cálculo de afinidad y comprobar que la salida por consola es idéntica a la obtenida dentro del IDE.
- En **Simulador de físicas 2D**. Compilar manualmente `MiProyecto.java` desde la terminal en `pr/src/` , ejecutar la simulación cinemática introduciendo valores de prueba y certificar la independencia del entorno de ejecución.
- En **Bóveda de contraseñas**. Compilar manualmente `MiProyecto.java` desde la terminal en `pr/src/` , verificar que la máquina virtual ejecuta la captura de credenciales y purgar el archivo `.class` del directorio de código fuente para mantener la higiene de Git.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). COMPILACIÓN DESACOPLADA CON DIRECTORIO DE DESTINO (`-d`) Y GESTIÓN DE CLASSPATH (`-cp`)

- **Contexto técnico.** En proyectos profesionales nunca se compilan los archivos `.class` mezclados dentro de la misma carpeta donde reside el código fuente (`src/`), ya que contamina el repositorio y dificulta el empaquetado.
- **Misión de la kata:**

a. En la raíz de tu proyecto, crea una carpeta llamada `bin/` destinada exclusivamente a almacenar binarios:

```
mkdir -p bin
```

b. Compila tu archivo fuente utilizando la bandera `-d` (*destination*) de `javac` para indicarle al compilador que deposite el archivo `.class` dentro de `bin/` sin tocar `pr/src/` :

```
javac -d bin pr/src/MiProyecto.java
```

c. Comprueba mediante `ls -la bin/` que el archivo `MiProyecto.class` se ha creado dentro de `bin/` y que `pr/src/` permanece completamente limpio.

d. Intenta ejecutar `java MiProyecto` desde la raíz. Comprobarás que la JVM emite un error de tipo `ClassNotFoundException` .

e. Soluciona el error utilizando el parámetro `-cp` (*Classpath*), indicándole a la máquina virtual dónde debe buscar los archivos compilados:

```
java -cp bin MiProyecto
```

f. Redacta dos líneas de comentario en tu libreta técnica explicando por qué ocurre esto: **la JVM necesita que el parámetro Classpath le indique la ruta exacta del sistema de archivos donde residen los paquetes y clases compiladas**.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). TELEMETRÍA DE LA JVM CON `-verbose:class` E INGENIERÍA INVERSA DE MEMORIA

- **Contexto de arquitectura.** Comprender cómo la Máquina Virtual de Java gestiona el ciclo de vida de carga de clases en memoria RAM y analizar la tabla de constantes del Bytecode mediante herramientas del JDK.

- **Misión de la kata:**

a. Ejecuta tu programa desde la terminal activando la bandera de telemetría detallada del cargador de clases (*ClassLoader*):

```
java -cp bin -verbose:class MiProyecto
```

b. Observa la avalancha de información en la consola antes de que aparezca tu primera línea de salida. Cuenta aproximadamente cuántas clases del sistema operativo y del núcleo de Java carga la JVM en la memoria RAM antes de llegar a ejecutar tu método `main` (descubrirás que carga más de 400 clases base).

c. Ejecuta el desensamblador oficial del JDK en modo exhaustivo (*verbose*):

```
javap -v bin/MiProyecto.class
```

d. Localiza en la salida el bloque `Constant Pool` **(Piscina de constantes)**. Identifica en qué líneas ha guardado Java los textos literales y constantes `final` que definiste en tu proyecto.

e. Anota en tu cuaderno qué valores tienen los campos `stack` (tamaño máximo de la pila de operandos de tu método `main`) y `locals` (número de ranuras de memoria reservadas para las variables locales).

f. Realiza una prueba de estrés de memoria simulando un entorno embebido ultrarrestringido, limitando el montículo (*heap*) de la JVM a solo 8 megabytes mediante la bandera de configuración de memoria:

```
java -cp bin -Xmx8m MiProyecto
```

3.1.2 Día 8 - 1 sesión

Teoría. Comparación de entornos de desarrollo

1. Caso guía en AzaharTech

Es viernes por la tarde en la sede de **AzaharTech**. En la pantalla de la sala de reuniones, **Laia Claramunt** proyecta una consulta técnica remitida por el equipo de soporte del **IES El Caminàs**:

*«Equipo, el coordinador de informática del centro nos traslada una situación real: en las aulas de informática disponen de equipos con IntelliJ IDEA, pero en los ordenadores portátiles prefieren utilizar la herramienta **Visual Studio Code**.*

Nos preguntan si abrir el proyecto con otro entorno de desarrollo diferente puede corromper el software o provocar incompatibilidades en el código.

*Hoy comprobaremos de forma práctica que un código bien estructurado no pertenece a un fabricante de software concreto. En vuestros proyectos de la bolsa (**Aventura conversacional**, **Motor de recomendación**, **Simulador de físicas 2D** o **Bóveda de contraseñas**) el núcleo de Java debe comportarse con absoluta fidelidad tanto si lo editamos en un entorno libre como en un entorno propietario».*

2. El ecosistema de herramientas de desarrollo

- **El IDE completo frente al editor modular extensible:**
- *IntelliJ IDEA Community.* Entorno de desarrollo integrado completo y libre diseñado específicamente para la máquina virtual de Java. Incluye de serie compilador, depurador, analizador de código y soporte nativo sin necesidad de plugins externos.
- *Visual Studio Code.* Editor de texto plano ligero y modular. Para trabajar con Java requiere instalar extensiones que se comuniquen con el JDK mediante el protocolo estándar de servidores de lenguaje (*Language Server Protocol - LSP*).
- **Software libre frente a software con telemetría/propietario:**
- IntelliJ IDEA Community cuenta con licencia de código abierto (*Apache 2.0*).

- VS Code utiliza una base libre (*VSCodium*), pero el instalador oficial de Microsoft incluye componentes y telemetría bajo licencia privativa.
- **Independencia del código fuente:**
- Ambos entornos utilizan el mismo compilador subyacente (`javac`) y la misma máquina virtual (`java`) instalados en el sistema operativo. El código no cambia; lo que cambia es la ergonomía y el consumo de recursos de la herramienta.

COMPARATIVA DE ARQUITECTURA DE ENTORNOS	
IntelliJ IDEA Community	Visual Studio Code
• Todas las funciones nativas • Licencia libre Apache 2.0	• Requiere Extension Pack for Java • Binario oficial con telemetría

---

Dojo de entrenamiento y katas de herramientas

Pasamos al dojo de entrenamiento de herramientas, con tres **katas de ejecución multi-entorno, análisis de rendimiento y compatibilidad de versiones** para entrenar la independencia técnica del software y la evaluación comparativa de herramientas:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). EJECUCIÓN DE TU PROYECTO PROPIO EN UN SEGUNDO ENTORNO (VS CODE)

- **Objetivo.** Abrir, compilar y ejecutar el archivo maestro de tu proyecto propio en **Visual Studio Code** (o en un editor modular secundario con el paquete de extensiones de Java), demostrando que la aplicación produce exactamente los mismos resultados en consola que en IntelliJ IDEA.
- **Aplicación según tu proyecto elegido:**
- En **Aventura conversacional**. Abrir la carpeta del proyecto en VS Code, comprobar que el editor reconoce el método `main`, ejecutar la captura de datos del personaje y verificar que los textos con caracteres especiales no se desconfiguran.
- En **Motor de recomendación**. Ejecutar el cálculo secuencial de afinidad dentro del panel de terminal de VS Code y certificar que la salida en pantalla coincide al céntimo con la obtenida en IntelliJ.
- En **Simulador de físicas 2D**. Comprobar la compilación de las variables cinemáticas en el entorno secundario y validar que el compilador no genera advertencias sintácticas.
- En **Bóveda de contraseñas**. Verificar la captura interactiva de datos de credenciales desde la consola integrada del nuevo entorno y confirmar que el código fuente permanece 100 % inalterado.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). AUDITORÍA DE CONSUMO DE RECURSOS Y HUELLA EN MEMORIA RAM

- **Contexto técnico.** Un desarrollador debe saber qué impacto tienen sus herramientas sobre el hardware del cliente para elegir el entorno idóneo según los recursos disponibles.
- **Misión de la kata:**

a. Con ambos entornos abiertos en tu equipo (IntelliJ IDEA y VS Code con el mismo proyecto cargado), abre el monitor del sistema operativo:

- En Windows: *Administrador de tareas* (`Ctrl + Shift + Esc`).
- En Linux: *Monitor del sistema* o comando `top` / `htop` en la terminal.

b. Localiza los procesos correspondientes a cada herramienta y anota los siguientes datos:

- Memoria RAM consumida por IntelliJ IDEA en reposo (habitualmente entre 800 MB y 1,5 GB).
- Memoria RAM consumida por VS Code con sus extensiones activas (habitualmente entre 300 MB y 600 MB).

c. Redacta dos líneas de conclusión técnica en tu libreta: **¿en qué escenarios reales del IES El Caminàs recomendarías instalar IntelliJ IDEA y en cuáles sería técnicamente más viable desplegar VS Code?**

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). DIAGNÓSTICO DEL ERROR `UnsupportedClassVersionError`

- **Contexto de arquitectura.** Comprender qué ocurre cuando un archivo `.class` compilado con una versión moderna de Java intenta ejecutarse en una máquina virtual antigua.
- **Misión de la kata:**

a. Cada versión del JDK asigna un número de versión mayor al Bytecode que genera (Java 8 = versión 52, Java 17 = versión 61, Java 21 = versión 65).

b. Ejecuta el desensamblador oficial del JDK para averiguar el número de versión interna de tu archivo compilado:

```
javap -v out/production/*/ControlAccesoQR.class | grep "major version"
```

(En Windows PowerShell: `javap -v out/production/Proyecto/ControlAccesoQR.class | Select-String "major version"`).

c. Comprueba que la salida muestra: `major version: 65` (correspondiente a Java 21).

d. Deduce y anota en tu cuaderno técnico: **¿qué error emitirá la JVM si intentamos ejecutar este archivo en un servidor del instituto que tenga instalado únicamente Java 17?** (Anota el nombre de la excepción: `java.lang.UnsupportedClassVersionError`).

3.2 Semana 5. Construcción y proyectos con Maven

3.2.1 Día 9 - 2 sesiones

Sesion 1. Teoría. El problema de la escala y el principio de convención sobre configuración

1. Caso guía en AzaharTech

Son las tres de la tarde del lunes 12 de octubre de 2026. En la sede de **AzaharTech**, el equipo de desarrollo inicia la segunda semana del Sprint 2.

Laia Claramunt proyecta en la pantalla el estado actual del repositorio del **IES El Caminàs**:

«La semana pasada aprendimos a compilar manualmente con `javac` y entendimos el papel de la máquina virtual. Sin embargo, en el mundo real una aplicación no se queda en un archivo único de 50 líneas. El sistema de acceso por QR va a incorporar paquetes con controladores de red, clases de validación de tokens y módulos de almacenamiento.

*En los proyectos de vuestros equipos ocurre lo mismo: **Aventura conversacional** necesitará estructurar escenas y personajes, **Motor de recomendación** dividirá su lógica de cálculo de afinidad de la gestión de ítems, **Simulador de físicas 2D** requerirá separar las fórmulas cinemáticas del motor gráfico, y **Bóveda de contraseñas** separará el evaluador de entropía del cifrado.*

*Si tuviéramos que compilar veinte clases a mano con la terminal indicando rutas una por una, pasaríamos más tiempo gestionando comandos que programando. Para solucionar esto, la industria utiliza **herramientas de automatización de la construcción (build tools)**. Hoy conoceremos el estándar mundial del ecosistema Java: **Apache Maven**».*

---

2. El porqué de Maven y la arquitectura estándar

¿QUÉ RESUELVE APACHE MAVEN?	
Problema tradicional	Solución estandarizada con Maven
Cada desarrollador crea carpetas con nombres distintos (bin, code...)	Convención sobre configuración: Estructura universal idéntica en cualquier empresa del mundo (src/main/java).
Compilación manual propensa a errores con	Ciclo de vida automatizado: Fases estándar para limpiar y compilar con

```
| comandos interminables.  | un solo clic (clean, compile).              |
└──────────────────────────┴─────────────────────────────────────────────┘
```

A. EL PRINCIPIO DE «CONVENCIÓN SOBRE CONFIGURACIÓN» (*CONVENTION OVER CONFIGURATION*)

Maven asume que todos los proyectos Java profesionales deben compartir la misma estructura física. Si el desarrollador respeta esa convención, no necesita escribir ningún script para explicarle al compilador dónde buscar el código:

```
proyecto-maven/
├── pom.xml                   <-- Archivo descriptor del proyecto
└── src/
    └── main/
        ├── java/             <-- Código fuente Java oficial (.java)
        └── resources/        <-- Archivos de configuración, textos e imágenes
```

B. EL ARCHIVO DESCRIPTOR `pom.xml` Y LAS COORDENADAS GAV

El corazón de un proyecto Maven es el archivo `pom.xml` (*Project Object Model*). Es un archivo en formato XML que define la identidad del software mediante tres coordenadas universales (**GAV**):

- **`groupId`.** Identificador de la organización o empresa, habitualmente en formato de dominio web invertido (*por ejemplo,* *`com.azahartech`*).
- **`artifactId`.** Nombre del proyecto o módulo concreto en minúsculas y separado por guiones (*por ejemplo,* *`control-acceso-caminas`*).
- **`version`.** Versión del desarrollo (*por ejemplo,* *`0.2.0-SNAPSHOT`* *para versiones en desarrollo*).

---

3. Actividad activa interactiva: Análisis y diseño del `pom.xml`

Los estudiantes analizan la sintaxis XML y definen en su libreta las coordenadas oficiales de su proyecto propio:

1. **Lectura de la plantilla mínima de Maven:**

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.azahartech.proyectos</groupId>
    <artifactId>nombre-de-tu-proyecto</artifactId>
    <version>0.2.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
</project>
```

2. **Diseño de coordenadas para los cuatro proyectos:**

- Cada estudiante anota en su libreta el bloque `<groupId>` y `<artifactId>` correspondiente a su reto de la bolsa.
- *Pregunta de análisis:* ¿Por qué especificamos `<maven.compiler.source>21</maven.compiler.source>`? Para garantizar que Maven rechace compilar si alguien utiliza una versión del JDK obsoleta en su máquina.

---

4. Puesta a punto de proyectos con estructura de directorios de Maven modificada

La estructura de directorios de los proyectos realizados por los equipos es diferente de la estructura básica de Maven.

Para que Maven pueda reconocer cada proyecto como propio, se debe añadir al pom.xml las siguientes líneas (dentro de la etiqueta project):

```xml
    <build>
        <sourceDirectory>pr/src/main/java</sourceDirectory>
        <resources>
            <resource>
                <directory>pr/src/main/resources</directory>
            </resource>
```

```
        </resources>
    </build>
```

Sesion 2. Dojo de entrenamiento y katas de herramientas

Pasamos al dojo de entrenamiento de herramientas, con tres **katas de construcción, estandarización de directorios y ciclo de compilación con Maven** para entrenar la transición del proyecto básico a la arquitectura profesional:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). INICIALIZACIÓN DEL PROYECTO MAVEN Y MIGRACIÓN DEL CÓDIGO FUENTE

- **Objetivo.** Crear el proyecto gestionado por Maven en IntelliJ IDEA dentro de tu espacio de trabajo oficial, configurar el archivo `pom.xml` mínimo para OpenJDK 21 y migrar la clase única de tu proyecto propio a la ruta estándar `src/main/java/`.
- **Aplicación según tu proyecto elegido:**
- En **Aventura conversacional**. Crear el proyecto Maven con el artifactId `aventura-conversacional`, ubicar `MiProyecto.java` en `src/main/java/` y verificar que el editor reconoce la nueva estructura estándar de carpetas.
- En **Motor de recomendación**. Configurar las propiedades de compilación para Java 21 en el `pom.xml`, mover el código a `src/main/java/` y sincronizar los cambios de Maven mediante el atajo `Ctrl + Shift + O`.
- En **Simulador de físicas 2D**. Inicializar el proyecto con Maven, estructurar la carpeta de recursos vacía `src/main/resources/` para futuras tablas de datos cinemáticos y comprobar que el código fuente compila sin errores.
- En **Bóveda de contraseñas**. Migrar el código del proyecto propio a la carpeta `src/main/java/` y verificar en el árbol de proyectos que IntelliJ asigna el icono de paquete azul oficial a la carpeta `java`.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). EL CICLO DE VIDA DE MAVEN: EJECUCIÓN VISUAL DE LAS FASES `clean` Y `compile`

- **Contexto técnico.** En proyectos profesionales no se compila invocando a `javac` manualmente; se utiliza el ciclo de vida de construcción estándar gestionado por el entorno.
- **Misión de la kata:**

a. En el margen derecho de IntelliJ IDEA, despliega la ventana lateral **Maven** (icono vertical de la 'm').

b. Localiza la sección **Lifecycle** del proyecto:

- Haz doble clic sobre la fase `compile`.
- Observa en la consola inferior cómo Maven ejecuta automáticamente el compilador y comprueba en el árbol de proyectos que se ha generado la carpeta `target/classes/` conteniendo el archivo compilado `.class`.

c. Haz doble clic sobre la fase `clean`:

- Comprueba en el árbol de proyectos cómo Maven elimina físicamente toda la carpeta `target/` de forma instantánea.

d. Ejecuta de forma combinada `clean` y a continuación `compile`.

e. Redacta dos líneas de conclusión en tu libreta técnica: **¿por qué es una buena práctica ejecutar la fase `clean` antes de una compilación importante?** (Para asegurar que no quedan archivos binarios residuales de clases eliminadas o renombradas).

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). AUDITORÍA DE DEPENDENCIAS INTERNAS Y DIRECTORIO LOCAL `.m2`

- **Contexto de arquitectura.** Comprender cómo Maven descarga y almacena en el disco duro del sistema operativo los plugins necesarios para compilar el código.

- **Misión de la kata:**

a. Abre la terminal integrada de IntelliJ (`Alt + F12`) en la raíz donde reside el archivo `pom.xml` .

b. Ejecuta una compilación forzando el modo de diagnóstico detallado (*debug*):

```
mvn compile -X
```

(En caso de no tener el comando global configurado, utiliza el menú contextual de Maven en IntelliJ: botón derecho sobre `compile` -> Run Maven Build with Debugging).

c. Observa la salida técnica de la consola: identifica las líneas donde Maven invoca al plugin oficial de compilación: `maven-compiler-plugin` .

d. Localiza en tu sistema operativo la carpeta oculta donde Maven guarda en caché todos los componentes descargados:

- En Windows: `C:\Users\tu-usuario\.m2\repository\`
- En Linux: `/home/tu-usuario/.m2/repository/`

e. Abre esa carpeta y comprueba cómo Maven almacena los plugins ordenados por carpetas según su `groupId` . Anota en tu cuaderno técnico: **¿por qué esta caché local evita tener que descargar las herramientas cada vez que creas un proyecto nuevo?**

3.2.2 Día 10 - 1 sesión

Teoria. Recursos del proyecto y búsqueda en Maven Central

1. Caso guía en AzaharTech

Es viernes por la tarde en **AzaharTech**. En la pantalla de pruebas, **Pau Ferrer** muestra la estructura del proyecto Maven que el equipo configuró el lunes pasado. La compilación de clases en `src/main/java/` funciona sin fallos, pero **Alba Torres** plantea un nuevo requerimiento del cliente:

*«El terminal del **IES El Caminàs** no solo va a ejecutar código Java puro. Necesitamos incluir archivos no ejecutables: un archivo de texto con el mensaje legal de bienvenida para el vestíbulo, los datos de configuración del centro y el logotipo institucional que se mostrará en pantalla.*

*¿Dónde se guardan estos archivos dentro de la estructura estándar de Maven para que no se pierdan al compilar? Y lo que es más importante: la semana que viene añadiremos librerías externas para generar códigos QR reales y procesar datos en vuestros proyectos (**Aventura conversacional**, **Motor de recomendación**, **Simulador de físicas 2D** y **Bóveda de contraseñas**). ¿De dónde saca Maven esas librerías?*

*Hoy aprenderemos a gestionar la carpeta **`src/main/resources/`** y descubriremos el gran almacén mundial de software del que se nutre nuestra herramienta: el repositorio **Maven Central**».*

---

2. Recursos del proyecto y el ecosistema Maven Central



A. LA CARPETA ESTÁNDAR `src/main/resources/`

En el desarrollo profesional, los archivos que no son código Java (imágenes, audios, archivos de texto, plantillas o configuraciones) jamás se mezclan con las clases dentro de `src/main/java/`:

- Se almacenan dentro del directorio reservado **`src/main/resources/`**.
- Cuando ejecutas la fase **`compile`** en Maven, la herramienta invoca automáticamente un plugin interno (`maven-resources-plugin`) que toma todo el contenido de `resources/` y lo copia de forma transparente dentro de **`target/classes/`**.
- De este modo, los recursos quedan empaquetados en la misma raíz que las clases y la JVM puede leerlos desde el *Classpath* sin importar en qué sistema operativo se ejecute la aplicación.

B. EL ALMACÉN MUNDIAL DE SOFTWARE: MAVEN CENTRAL

¿Cómo sabe Maven de dónde descargar una librería cuando la necesitamos?

- **Maven Central (`repo.maven.apache.org`).** Es el repositorio público oficial en la nube mantenido por la comunidad de software donde empresas y desarrolladores de todo el mundo publican sus librerías de Java certificadas.
- Para localizar cualquier librería y conocer sus coordenadas exactas (**GAV**: `groupId`, `artifactId`, `version`), los desarrolladores consultamos el portal indexador oficial: **mvnrepository.com**.

Dojo de entrenamiento y katas de herramientas

Pasamos al dojo de entrenamiento de herramientas, con tres **katas de gestión de recursos, exploración en Maven Central y filtrado de propiedades** para entrenar la arquitectura de proyectos industriales:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). INTEGRACIÓN DE RECURSOS DEL PROYECTO PROPIO EN `src/main/resources/`

- **Objetivo.** Crear un archivo de recursos plano dentro de la ruta estándar `src/main/resources/`, compilar el proyecto con Maven y comprobar que el archivo se transfiere automáticamente a `target/classes/` junto al Bytecode compilado.
- **Aplicación según tu proyecto elegido:**
- En **Aventura conversacional**. Crear el archivo `src/main/resources/bienvenida.txt` con la ambientación introductoria del juego, ejecutar la fase `compile` desde el panel de Maven y verificar en el árbol de proyectos que el archivo de texto aparece duplicado en `target/classes/bienvenida.txt`.
- En **Motor de recomendación**. Crear el archivo `src/main/resources/categorias.txt` con el catálogo base de géneros de ocio y validar que Maven gestiona el recurso sin errores de compilación.
- En **Simulador de físicas 2D**. Crear el archivo `src/main/resources/constantes-universo.properties` definiendo valores de gravedad y fricción simulada, comprobando su despliegue en la carpeta de clases de salida.
- En **Bóveda de contraseñas**. Crear el archivo `src/main/resources/politicas-seguridad.txt` con los requisitos de longitud y caracteres para las contraseñas, verificando la sincronización de recursos con Maven.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). EXPLORACIÓN TÉCNICA DE DEPENDENCIAS EN MAVEN CENTRAL

- **Contexto técnico.** Un desarrollador de AzaharTech no descarga archivos `.jar` sueltos desde páginas web dudosas; busca las coordenadas oficiales en el repositorio central de Maven para garantizar la seguridad y autenticidad del software.
- **Misión de la kata:**
- Abre tu navegador web y accede al portal oficial de búsqueda de dependencias: **https://mvnrepository.com**.
- Localiza una librería estándar del ecosistema Java representativa según la temática de tu proyecto propio:
- Para **Aventura conversacional** o **Bóveda de contraseñas**: busca la librería **`JSON-java`** (coordenadas de la organización `org.json`).
- Para **Motor de recomendación**: busca la librería de utilidades de colecciones y cadenas **`Apache Commons Lang`** (organización `org.apache.commons`).
- Para **Simulador de físicas 2D**: busca la librería matemática y de matrices **`Apache Commons Math`** (organización `org.apache.commons`).

- Selecciona la versión estable más reciente y haz clic sobre la pestaña de configuración para **Maven**.
- Copia en tu libreta técnica el bloque XML `<dependency>` resultante, identificando claramente cuáles son sus tres coordenadas (`groupId`, `artifactId` y `version`).
- *Pregunta de investigación:* ¿Qué diferencia hay entre una versión marcada como *Release* y una marcada como *SNAPSHOT* o *Alpha*? Anota en tu cuaderno por qué en producción solo debemos usar versiones estables *Release*.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). FILTRADO AUTOMÁTICO DE VARIABLES (*RESOURCE FILTERING*) EN EL `pom.xml`

- **Contexto de arquitectura.** Automatizar que Maven inyecte información de la versión del proyecto directamente dentro de un archivo de texto en tiempo de compilación sin escribirla a mano.
- **Misión de la kata:**
- En tu archivo `src/main/resources/info.txt`, escribe el siguiente contenido utilizando la sintaxis de variables de Maven:

```
Aplicación: ${project.artifactId}
Versión oficial: ${project.version}
Entorno de compilación: Java ${maven.compiler.target}
```

- Si compilas ahora, Maven copiará el archivo tal cual, mostrando literalmente el texto `${project.version}`.
- Para activar la sustitución dinámica de variables, abre tu archivo `pom.xml` y añade dentro de la etiqueta `<build>` la instrucción de filtrado de recursos:

```
<build>
    <resources>
        <resource>
            <directory>src/main/resources</directory>
            <filtering>true</filtering>
        </resource>
    </resources>
</build>
```

- Sincroniza los cambios de Maven en IntelliJ (`Ctrl + Shift + O`) y ejecuta la fase `compile`.
- Abre el archivo resultante en `target/classes/info.txt` y comprueba la magia de la automatización: Maven habrá reemplazado las variables por los valores reales de tu proyecto (*por ejemplo,* `Versión oficial: 0.2.0-SNAPSHOT`).

3.3 Semana 6. Gestión de dependencias y cierre

3.3.1 Día 11 - 2 sesiones

Sesión 1. Teoría. De la nada al código QR: Maven Central, dependencias declarativas y renderizado en memoria

---

1. Caso guía en AzaharTech

Son las tres de la tarde del lunes 19 de octubre de 2026. Entramos en la última semana del **Sprint 2** en la sede de **AzaharTech** en Castellón de la Plana.

En la pantalla de proyección, **Laia Claramunt** y **Alba Torres** muestran el nuevo requerimiento técnico enviado por la dirección del **IES El Caminàs**:

«Equipo, el prototipo del instituto ya compila con la estructura estándar de Maven y gestiona limpiamente su ciclo de construcción. Pero hoy el cliente nos plantea un salto funcional determinante:

El terminal de conserjería debe generar en pantalla el código QR del carné del estudiante para que el lector óptico del torno o el conserje puedan verificar su identidad en el acceso al centro.

Programar un algoritmo de código QR desde cero requeriría meses implementando codificación binaria, máscaras de píxeles y corrección de errores Reed-Solomon. En la ingeniería de software profesional nadie reinventa la rueda.

*Para eso existe **Maven Central**: un repositorio público mundial con millones de librerías abiertas. Con solo declarar cuatro líneas en el archivo `pom.xml`, Maven descargará la librería industrial **ZXing Core** de Google, resolverá sus dependencias y nos permitirá generar una matriz QR real.*

Como todavía no hemos estudiado la gestión de ficheros en disco (que abordaremos más adelante en el curso), no guardaremos imágenes en carpetas: utilizaremos los bucles y condicionales que estáis aprendiendo en Programación para renderizar el código QR directamente en la consola con caracteres de texto. ¡Y será tan nítido que podréis escanearlo con la cámara de vuestro propio teléfono móvil apuntando a la pantalla del monitor!».

---

2. El infierno de las librerías frente a la gestión declarativa de Maven

Antes de la llegada de herramientas de construcción como Apache Maven, incorporar código de terceros era un proceso manual, frágil y propenso a errores conocido como el **Jar Hell** (*el infierno de los JARs*):

```
┌──────────────────────────────────────────────────────────────────────┐
│                    EVOLUCIÓN EN LA GESTIÓN DE LIBRERÍAS              │
├──────────────────────────────────┬───────────────────────────────────┤
│ Método artesanal antiguo         │ Gestión declarativa con Maven     │
├──────────────────────────────────┼───────────────────────────────────┤
│ 1. Buscar y descargar archivos   │ 1. Declarar coordenadas GAV en el │
│    .jar en páginas web diversas. │    archivo pom.xml.               │
│ 2. Copiarlos a mano en carpetas  │ 2. Maven descarga y valida        │
│    locales del proyecto.         │    automáticamente los binarios   │
│ 3. Si la librería necesita otra  │    desde Maven Central.           │
│    librería secundaria, buscarla │ 3. Resolución automática del árbol │
│    manualmente a ciegas.         │    completo de dependencias       │
│ 4. Errores de ClassNotFoundException│   transitivas en ~/.m2/repository.│
│    en tiempo de ejecución.       │ 4. Integración transparente en el │
│                                  │    Classpath del IDE.             │
└──────────────────────────────────┴───────────────────────────────────┘
```

COORDENADAS GAV Y DEPENDENCIAS TRANSITIVAS

Para solicitar una librería a Maven Central, únicamente debemos indicar sus tres coordenadas universales: **GroupId**, **ArtifactId** y **Version**:

- **Dependencia directa:** Aquella que declaras explícitamente en el bloque `<dependencies>` de tu `pom.xml`.
- **Dependencia transitiva:** Si la librería que has solicitado necesita a su vez otras librerías auxiliares para operar, Maven analiza el descriptor remoto de esa librería y descarga toda la cadena de dependencias de forma automática.

```
RESOLUCIÓN DECLARATIVA EN MAVEN:

   Tu pom.xml ──────► Solicita: [com.google.zxing:core:3.5.3]
                              │
                              ▼
                      Maven Central
                              │ (Descarga automática a caché local)
                              ▼
                 ~/.m2/repository/com/google/zxing/core/3.5.3/core-3.5.3.jar
                              │
                              ▼
                Classpath de tu aplicación (Listo para usar en Java)
```

---

3. El caso guía en acción: Renderizado de QR en memoria y consola

Pau Ferrer abre IntelliJ IDEA y muestra el procedimiento completo para dotar al proyecto del IES El Caminàs de su generador de códigos QR sin tocar el sistema de archivos:

PASO 1: DECLARAR LA DEPENDENCIA EN EL `pom.xml`

Abrimos el archivo de configuración de Maven y añadimos el bloque de dependencias:

```
<dependencies>
    <!-- Motor de cálculo y generación de matrices QR de Google -->
    <dependency>
        <groupId>com.google.zxing</groupId>
        <artifactId>core</artifactId>
        <version>3.5.3</version>
    </dependency>
</dependencies>
```

PASO 2: SINCRONIZAR MAVEN EN INTELLIJ IDEA

Tras editar el archivo, pulsamos el botón flotante con el icono de Maven (o el atajo `Ctrl + Shift + O`). IntelliJ contacta con Maven Central, descarga el archivo JAR en milisegundos y lo añade a las librerías del proyecto.

PASO 3: DIBUJAR LA MATRIZ EN LA CONSOLA

Creamos la clase `GeneradorQRCaminas`. La librería externa realiza los cálculos matemáticos avanzados y nos entrega una matriz bidimensional de bits (`BitMatrix`). Nosotros nos limitamos a recorrerla con dos bucles anidados e imprimir caracteres de bloque en pantalla:

```
package es.azahartech.caminas;

import com.google.zxing.BarcodeFormat;
import com.google.zxing.common.BitMatrix;
import com.google.zxing.qrcode.QRCodeWriter;

public class GeneradorQRCaminas {

    public static void main(String[] args) throws Exception {
        String datosAlumno = "ALUMNO:12004217;CENTRO:IES_EL_CAMINAS;CURSO:1DAM";

        // 1. Invocamos a la librería externa para calcular la matriz QR
        QRCodeWriter escritorQR = new QRCodeWriter();
        // Generamos una matriz de 25x25 módulos en memoria
        BitMatrix matriz = escritorQR.encode(datosAlumno, BarcodeFormat.QR_CODE, 25, 25);

        System.out.println("================================================");
        System.out.println("  CÓDIGO QR EN CONSOLA - ACCESO IES EL CAMINÀS   ");
        System.out.println("  Apunta con la cámara de tu móvil a la pantalla ");
        System.out.println("================================================\n");

        // 2. Recorrido bidimensional de la matriz en memoria RAM (sin ficheros)
        for (int y = 0; y < matriz.getHeight(); y++) {
            for (int x = 0; x < matriz.getWidth(); x++) {
                // Si la posición tiene bit activo pintamos doble bloque negro "██";
                // si no, pintamos dos espacios en blanco "  "
                if (matriz.get(x, y)) {
                    System.out.print("██");
                } else {
                    System.out.print("  ");
                }
            }
            System.out.println(); // Salto a la siguiente fila
        }

        System.out.println("\n[OK] Código QR generado en memoria RAM y renderizado con éxito.");
    }
}
```

Al pulsar **Run (`Shift + F10`)**, la terminal de IntelliJ se transforma en un panel gráfico de texto: los estudiantes enfocan la consola con sus teléfonos móviles y la cámara reconoce al instante el texto codificado del instituto.

Sesión 2. Dojo de entrenamiento y katas de herramientas

Pasamos a los puestos individuales. Cada alumno integrará una librería externa especializada de Maven Central en su proyecto de la bolsa para incorporar funciones profesionales sin necesidad de manipular ficheros:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). EL GENERADOR QR EN TU ENTORNO LOCAL

- **Objetivo.** Integrar la dependencia `zxing:core` en el proyecto del caso guía, sincronizar el proyecto con Maven Central y renderizar un código QR funcional en la consola de tu equipo.
- **Misión:**

a. Abre el archivo `pom.xml` e introduce el bloque `<dependencies>` con `com.google.zxing:core:3.5.3`.

b. Sincroniza el gestor con **`Ctrl + Shift + O`**.

c. Implementa la clase de renderizado en `src/main/java/` y ejecuta el programa.

d. Comprueba que el código QR aparece perfectamente cuadrado en la consola inferior de IntelliJ. Si se ve deformado, ajusta el ancho de los caracteres utilizando el doble bloque `"██"`.

e. Escanéalo con tu propio móvil y verifica que lee los datos del alumno correctamente.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). INYECCIÓN DE SUPERPODERES MAVEN EN TU PROYECTO PROPIO

- **Contexto técnico.** En la industria del software seleccionamos librerías consolidadas para dotar a nuestros programas de capacidades avanzadas en memoria sin escribir código redundante.
- **Misión según tu proyecto de la bolsa:**
- **En Aventura conversacional:**
- *Librería:* **`com.github.freva:ascii-table:1.8.0`** (Generador de tablas formateadas en consola).
- *Aplicación:* Declara la dependencia en tu `pom.xml` y utiliza su API para sustituir los textos planos del menú por una tabla alineada en consola que muestre las estadísticas del personaje (Nivel, Puntos de Vida `HP`, Puntos de Magia `MP` y Oro) con bordes dobles profesionales.
- **En Motor de recomendación:**
- *Librería:* **`org.apache.commons:commons-math3:3.6.1`** (Matemáticas avanzadas de Apache).
- *Aplicación:* Declara la dependencia en tu `pom.xml` e instancia dos objetos `ArrayRealVector` para representar las valoraciones numéricas de dos usuarios. Utiliza el método `.cosine(otroVector)` para calcular la similitud coseno entre ambos en una sola línea de código e imprime el porcentaje de afinidad por consola.
- **En Simulador de físicas 2D:**
- *Librería:* **`org.joml:joml:1.10.5`** (Álgebra vectorial en memoria).
- *Aplicación:* Declara la dependencia y sustituye las variables primitivas de posición y aceleración por un objeto `Vector2f`. Aplica la aceleración sobre la velocidad mediante el método vectorial `velocidad.add(aceleracion)` e imprime las nuevas coordenadas resultantes en la consola.
- **En Bóveda de contraseñas:**
- *Librería:* **`org.mindrot:jbcrypt:0.4`** (Algoritmo de cifrado y derivación de claves BCrypt).
- *Aplicación:* Declara la dependencia y utiliza `BCrypt.hashpw(passwordPlana, BCrypt.gensalt(12))` para generar en memoria un hash criptográfico real de 60 caracteres. Imprime el resultado en consola y comprueba mediante `BCrypt.checkpw()` cómo la librería valida si una contraseña introducida coincide o no sin necesidad de descifrarla.

- **Misión alternativa según tu proyecto de la bolsa:**
- **En Aventura conversacional:**
- *Librería:* `com.diogonunes.JColor:5.5.1` (Colores en el texto).
- *Aplicación:* Colorear las estadísticas fijas del héroe: la salud en verde brillante, el nombre del enemigo en rojo y las monedas en amarillo.
- **En Motor de recomendación:**
- *Librería:* `net.datafaker.datafaker:2.4.2` (Generar datos del mundo real).
- *Aplicación:* Generar automáticamente un título de película ficticia con faker.book().title() y una valoración decimal aleatoria para calcular su afinidad.
- **En Simulador de físicas 2D:**
- *Librería:* `com.diogonunes.JColor:5.5.1` (Colores en el texto).
- *Aplicación:* Destacar los vectores cinemáticos en la salida formateada: velocidad inicial en azul y aceleración gravitatoria en magenta.
- **En Bóveda de contraseñas:**
- *Librería:* `net.datafaker.datafaker:2.4.2` (Generar datos del mundo real).
- *Aplicación:* Instanciar faker.internet().password(12, 16, true, true) para generar una contraseña candidata de alta entropía y medir su longitud con métodos de String.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). RADIOGRAFÍA DE MAVEN CENTRAL Y CONFIRMACIÓN VISUAL EN GIT

- **Contexto de arquitectura.** Un desarrollador senior debe auditar físicamente dónde residen los binarios descargados y confirmar las modificaciones en el repositorio respetando las buenas prácticas del equipo.
- **Misión:**

a. **Inspección de la caché local (.m2):**

- Localiza en el explorador de archivos de tu sistema operativo la carpeta oculta de Maven en tu directorio de usuario: `~/.m2/repository/` (o `%USERPROFILE%\.m2\repository\` en Windows).
- Navega hasta la carpeta correspondiente a tu librería (por ejemplo, `com/google/zxing/core/3.5.3/`) y comprueba que Maven ha descargado el archivo `.jar` compilado y su archivo descriptor `.pom`.

b. **Auditoría visual de dependencias en IntelliJ:**

- Abre la pestaña lateral derecha **Maven**.
- Despliega la rama **Dependencies** de tu proyecto y confirma que la librería aparece con su icono azul vinculada al proyecto.

c. **Commit visual en IntelliJ IDEA (sin consola):**

- Pulsa `Ctrl + K` (o abre la pestaña lateral izquierda *Commit*).
- En el visor de diferencias (*Diff Viewer*), haz doble clic sobre `pom.xml` para comprobar visualmente que solo se han añadido las etiquetas de dependencias.
- Marca con visto bueno (`[x]`) los archivos modificados (`pom.xml` y la nueva clase Java).
- Escribe en el cuadro de texto el mensaje estructurado: `feat(core): incorporada libreria externa maven y renderizado en memoria`
- Haz clic en el botón azul **Commit**.

3.3.2 Día 12 - 1 sesión

Taller de consolidación: ciclo de vida de Maven, higiene de repositorio y cierre formal del Sprint 2

---

1. Caso guía en AzaharTech

Viernes 23 de octubre de 2026. Son las cuatro de la tarde en la oficina de **AzaharTech** en Castellón de la Plana. La jornada marca el cierre definitivo del **Sprint 2: «El viaje del código: compilación, máquinas virtuales y dependencias»**.

En las pantallas de los puestos de desarrollo, el generador de códigos QR del **IES El Caminàs** y los cuatro proyectos singulares de la bolsa (**Aventura conversacional**, **Motor de recomendación**, **Simulador de físicas 2D** y **Bóveda de contraseñas**) se ejecutan con fluidez: cada uno incorpora ya librerías externas de primer nivel gestionadas por Maven Central y renderizadas limpiamente en memoria RAM sin necesidad de ensuciar el disco con ficheros temporales.

Laia Claramunt y **Pau Ferrer** reúnen al equipo frente al proyector para la sesión de cierre:

«Equipo, en este segundo sprint habéis conquistado un terreno técnico crucial: sabéis cómo el compilador genera bytecode, cómo la JVM lo interpreta en cualquier plataforma y cómo Maven descarga y enlaza dependencias industriales en cuestión de segundos.

Sin embargo, antes de dar un sprint por completado en AzaharTech, todo software debe superar dos controles de calidad indispensables: 1. *Validar que el ciclo de vida de Maven ejecuta las fases de limpieza y compilación sin depender de configuraciones manuales del IDE.* 2. *Auditar la higiene estricta de Git: verificar que la carpeta* `target/` *y los metadatos internos del sistema jamás contaminen el repositorio remoto en GitHub.*

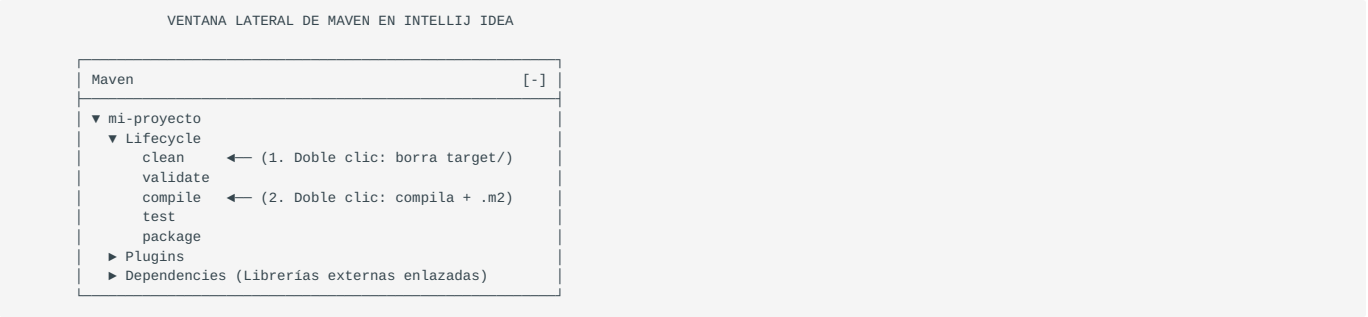
Hoy cerraremos formalmente el Sprint 2 ejecutando el ciclo de vida de Maven desde la interfaz de IntelliJ IDEA y realizando el commit y push visual definitivo antes de encarar el lunes el Sprint 3: el laberinto de la lógica, las pruebas formales y la depuración interactiva».

---

2. La prueba de fuego: ciclo de vida visual de Maven en IntelliJ

Cuando un proyecto integra librerías externas, la compilación no se limita a invocar al compilador `javac`: Maven debe verificar la integridad de las dependencias alojadas en `~/.m2/repository` y enlazarlas en el *Classpath* de compilación.

Para comprobar que el proyecto compila de forma reproducible y autónoma, los desarrolladores ejecutan las fases del ciclo de vida utilizando la **ventana lateral de Maven** de IntelliJ IDEA:



VERIFICACIÓN EN DOS PASOS:

1. **Fase `clean`:**
   - Haz doble clic sobre **clean** dentro del menú *Lifecycle*.
   - Observa cómo la carpeta de salida `target/` desaparece instantáneamente del árbol del proyecto, garantizando que no queden restos de compilaciones previas.
2. **Fase `compile`:**
   - Haz doble clic sobre **compile**.
   - Maven analiza el archivo `pom.xml`, valida las dependencias externas (como `zxing-core`), compila todo el código fuente de `src/main/java/` y genera de nuevo la estructura binaria en `target/classes/`.
   - La consola inferior de Maven debe culminar con el mensaje en verde: `[INFO] BUILD SUCCESS`.

---

3. Auditoría de higiene del repositorio: el archivo `.gitignore`

Uno de los errores más graves de un desarrollador júnior es versionar en Git los archivos binarios compilados o los archivos de configuración local de su propio ordenador.

En un proyecto gestionado con Maven, el código fuente (`src/`) y la receta de construcción (`pom.xml`) son los **únicos** elementos necesarios para reconstruir el software en cualquier máquina del mundo.

```
┌──────────────────────────────────────────────────────────────────┐
│                  MAPA DE HIGIENE TÉCNICA DEL REPOSITORIO         │
├──────────────────────────┬──────────────┬────────────────────────┤
│ Elemento / Directorio    │ ¿Va a Git?   │ Justificación técnica  │
├──────────────────────────┼──────────────┼────────────────────────┤
│ src/                     │ SÍ           │ Código fuente y recursos.  │
│ pom.xml                  │ SÍ           │ Declaración del proyecto.  │
│ README.md                │ SÍ           │ Documentación del software.│
├──────────────────────────┼──────────────┼────────────────────────┤
│ target/                  │ NUNCA        │ Bytecode autogenerado.     │
│ .idea/ y *.iml           │ NUNCA        │ Configuración de tu IDE.   │
│ .DS_Store / Thumbs.db    │ NUNCA        │ Basura del sistema operativo│
└──────────────────────────┴──────────────┴────────────────────────┘
```

INSPECCIÓN VISUAL EN EL ÁRBOL DE INTELLIJ IDEA

Cada estudiante abre el archivo `.gitignore` situado en la raíz del proyecto y confirma que contiene las exclusiones estándar de la empresa:

```
# Directorio de construcción y binarios de Maven
target/

# Metadatos del entorno de desarrollo (IntelliJ IDEA)
.idea/
*.iml
out/

# Archivos de índices del sistema operativo
.DS_Store
Thumbs.db
```

Comprobación visual de Pau Ferrer: En el panel del explorador de proyectos de IntelliJ, la carpeta `target/` debe mostrarse con una tipografía de **color naranja oscuro o gris atenuado** (*ignored*). Si aparece en color blanco, azul o rojo, Git la está vigilando por error y debe regularizarse de inmediato antes del commit.

---

4. Cierre formal del Sprint 2 mediante Commit y Push visual

Siguiendo el estándar pedagógico de AzaharTech, el cierre del sprint se formaliza **exclusivamente a través de la interfaz gráfica de IntelliJ IDEA**, sin recurrir a la terminal de comandos:

```
                VENTANA VISUAL DE COMMIT EN INTELLIJ IDEA (Ctrl + K)

┌──────────────────────────────────────────────────────────────────┐
│ Commit: Local Changes                                            │
├──────────────────────────────────────────────────────────────────┤
│ Changes (2 files)                                                │
│  [x] pom.xml                      (dependencia zxing/externa)    │
│  [x] src/main/java/.../App.java   (renderizado de QR / lógica)   │
├──────────────────────────────────────────────────────────────────┤
│ Mensaje de confirmación:                                         │
│ ┌──────────────────────────────────────────────────────────────┐ │
│ │ feat(core): integradas dependencias maven y renderizado en memoria │ │
│ └──────────────────────────────────────────────────────────────┘ │
│                                                                  │
│ [ Commit v ]  ──► Desplegar flecha: [ Commit and Push... ]       │
└──────────────────────────────────────────────────────────────────┘
```

PASOS VISUALES PASO A PASO:

1. **Abrir la herramienta visual de cambios:**

- Pulsa el atajo universal **`Ctrl + K`** (en Windows/Linux) o haz clic en la pestaña vertical izquierda **Commit** (`Alt + 0`).

2. **Revisión de diferencias (*Diff Viewer*):**
   - Haz doble clic sobre `pom.xml` en la lista de cambios. Comprueba en el panel comparativo que las etiquetas `<dependency>` se han introducido de forma limpia.
   - Haz doble clic sobre la clase Java para certificar que el código recorre la matriz en memoria sin dependencias de ficheros.
3. **Selección de archivos:**
   - Marca con visto bueno (`[x]`) únicamente los archivos modificados que deben formar parte del repositorio (`pom.xml` , `.gitignore` y clases Java bajo `src/main/java/`).
   - Cerciórate de que ningún fichero dentro de `target/` aparece seleccionado.
4. **Redacción del mensaje de commit:**
   - Escribe en el cuadro de texto el mensaje formal de cierre: `feat(core): integradas dependencias maven y renderizado en memoria`
5. **Confirmación y sincronización con GitHub (*Commit and Push*):**
   - Haz clic sobre la flecha desplegable del botón azul y selecciona **Commit and Push...** (o pulsa `Ctrl + Alt + K`).
   - En la ventana emergente de confirmación de subida (*Push Commits*), revisa que la rama de destino sea tu repositorio personal en GitHub y haz clic en el botón azul **Push**.

> Al mostrarse la notificación emergente verde en la esquina inferior derecha («*Pushed 1 commit to origin/main*»), el **Sprint 2: «El viaje del código: compilación, máquinas virtuales y dependencias»** queda oficialmente completado y sellado.
>
> El equipo domina la construcción automatizada, el enlace de librerías de Maven Central y la higiene del repositorio digital. El próximo lunes arrancará el **Sprint 3**, donde utilizaremos el depurador interactivo de IntelliJ para aprender a congelar la ejecución y cazar los errores de nuestras estructuras de control algorítmicas.

4. Sprint 3. El ciclo del error: pruebas y depuración

4.1 Semana 7. Depuracion interactiva e inspección de memoria

4.1.1 Día 13 - 2 sesiones

Sesión 1. Teoría. La radiografía del código en ejecución: del print casero a la depuración interactiva

1. Caso guía en AzaharTech

Son las tres de la tarde del lunes 26 de octubre de 2026. En la sede de **AzaharTech** en Castellón de la Plana, arranca formalmente la primera jornada del **Sprint 3**.

Tras haber consolidado la gestión de paquetes, compilación y dependencias con Maven en el Sprint anterior, el equipo se reúne en la sala principal. En la pantalla de proyección, **Laia Claramunt** muestra una captura del panel de incidencias enviado de urgencia a primera hora de la mañana por el coordinador TIC del **IES El Caminàs**:

«Equipo, entramos de lleno en el corazón de la lógica de programación. Durante estas semanas, en el módulo de Programación estáis implementando estructuras de control complejas: bucles anidados (`for`, `while`), bifurcaciones condicionales (`if`/`else`, `switch`) y control de excepciones.

La aplicación del IES El Caminàs ya cuenta con su primer prototipo de validación de códigos QR en bucle continuo para procesar la cola de alumnos en el vestíbulo. Sin embargo, esta mañana el sistema ha entrado en un bloqueo total: tras procesar al décimo alumno, la aplicación se congela consumiendo el 100 % de la CPU de los terminales de conserjería, o bien asigna faltas injustificadas a estudiantes que sí habían registrado el código.

El impulso natural de un programador principiante es llenar el código de sentencias `System.out.println("Pasa por aquí 1")`, `System.out.println("Variable x = " + x)` y `System.out.println("Pasa por aquí 2")`. En la industria del software profesional, esa práctica es un antipatrón ineficiente, peligroso y que ensucia el repositorio.

*Tanto en el caso del IES El Caminàs como en los proyectos singulares de vuestros equipos (**Aventura conversacional**, **Motor de recomendación**, **Simulador de físicas 2D** o **Bóveda de contraseñas**), hoy aprenderemos a congelar el tiempo: usaremos el depurador interactivo (Debugger) de IntelliJ IDEA para pausar la ejecución de la JVM, examinar la memoria RAM en vivo y alterar el comportamiento del programa sin tocar una sola línea de código fuente».*

2. La naturaleza de los errores y las limitaciones del «Print Debugging»

En el ciclo de vida del software, encontrar errores no es una anomalía, sino una certeza estadística. Sin embargo, para erradicarlos con rigor técnico, primero debemos categorizarlos:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        TAXONOMÍA DE ERRORES EN EL SOFTWARE               │
├──────────────────────┬──────────────────────────────┬────────────────────┤
│ Tipo de error        │ ¿Cuándo se manifiesta?       │ Comportamiento / Detección │
├──────────────────────┼──────────────────────────────┼────────────────────┤
│ 1. De compilación    │ Durante la fase de construcción │ El compilador (javac) detecta │
│    (Sintaxis/Tipos)  │ con Maven o en el propio IDE.  │ faltas sintácticas y no genera │
│                      │                              │ el archivo .class.  │
├──────────────────────┼──────────────────────────────┼────────────────────┤
│ 2. De ejecución      │ Con el programa ya en marcha │ La JVM interrumpe el flujo │
│    (Runtime / Crash)│ dentro de la máquina virtual.  │ lanzando una excepción fatal │
│                      │                              │ (NullPointerException, etc.). │
├──────────────────────┼──────────────────────────────┼────────────────────┤
│ 3. Lógico            │ El programa compila y corre sin │ El software hace exactamente │
│    (Semántico)       │ romperse, pero da un resultado  │ lo que se escribió, pero no lo │
```

```
|                     | funcional incorrecto o cuelgue. | que el negocio requería.       |
└─────────────────────┴─────────────────────────────────┴────────────────────────────────┘
```

EL PROBLEMA DEL «PRINT DEBUGGING»

Llenar el código con instrucciones de consola para rastrear variables presenta graves desventajas de ingeniería: 1. **Contaminación de código:** Obliga a modificar el archivo fuente, rompiendo la trazabilidad limpia de commits en Git. 2. **Coste temporal:** Requiere recompilar con Maven y relanzar la JVM cada vez que queremos observar una variable diferente. 3. **Pérdida de contexto:** Un `System.out.println` muestra una foto aislada en texto plano, pero oculta la pila de llamadas (*Call Stack*), los objetos en segundo plano y el estado global de la memoria. 4. **Riesgo en producción:** Olvidar trazas de depuración en despliegues reales puede filtrar contraseñas, tokens y saturar los ficheros de log de la empresa.

```
COMPARATIVA METODOLÓGICA:

A) Depuración primitiva (Print Debugging):
   [Modificar código] ──► [Recompilar] ──► [Ejecutar] ──► [Leer consola] ──► [Borrar print]
   (Ciclo lento, destructivo y ciego ante bucles complejos)

B) Depuración interactiva profesional (IDE Debugger):
   [Poner Breakpoint] ──► [Lanzar Debug] ──► [Congelar JVM] ──► [Inspeccionar / Modificar RAM]
   (Sin tocar código, inspección multidimensional e inmediata)
```

---

3. Anatomía del depurador interactivo en IntelliJ IDEA

Cuando lanzamos nuestro software mediante el botón de depuración (**Debug** o `Shift + F9`), IntelliJ no invoca a la JVM de forma estándar: la arranca comunicándose mediante el protocolo **JDWP** (*Java Debug Wire Protocol*). Esto permite al IDE tomar el control absoluto del hilo de ejecución.

```
┌──────────────────────────────────────────────────────────────────────┐
│                  MANDOS DEL DEPURADOR EN INTELLIJ IDEA               │
├──────────────┬──────────────┬────────────────────────────────────────┤
│ Acción       │ Atajo        │ Propósito técnico                      │
├──────────────┼──────────────┼────────────────────────────────────────┤
│ Step Over    │ F8           │ Avanza a la siguiente línea del método │
│              │              │ actual sin entrar dentro de funciones. │
├──────────────┼──────────────┼────────────────────────────────────────┤
│ Step Into    │ F7           │ Entra al interior del método invocado  │
│              │              │ en la línea actual para ver su lógica. │
├──────────────┼──────────────┼────────────────────────────────────────┤
│ Step Out     │ Shift + F8   │ Sale del método actual y regresa a la  │
│              │              │ línea del método llamador.             │
├──────────────┼──────────────┼────────────────────────────────────────┤
│ Resume       │ F9           │ Reanuda la ejecución libre hasta topar │
│ Program      │              │ con el siguiente punto de ruptura.     │
├──────────────┼──────────────┼────────────────────────────────────────┤
│ Evaluate     │ Alt + F8     │ Abre una consola para evaluar fórmulas y │
│ Expression   │              │ métodos en caliente contra la RAM real.  │
└──────────────┴──────────────┴────────────────────────────────────────┘
```

PUNTOS DE RUPTURA (*BREAKPOINTS*)

- **Punto de ruptura estándar:** Se activa haciendo clic en el canal lateral izquierdo (*gutter*) junto al número de línea. Marca una línea donde la JVM suspenderá el hilo de ejecución justo **antes** de procesarla.
- **Punto de ruptura condicional:** Clic derecho sobre el punto rojo del breakpoint. Permite definir una expresión lógica booleana en Java (por ejemplo, `alumnoId == 10` o `intentos > 3`). El programa solo se detendrá cuando la condición sea verdadera, evitando pulsar manualmente *Step Over* cientos de veces en bucles largos.

---

4. Actividad activa interactiva: Caza visual del bug en la pizarra

Alba Torres y Pau Ferrer proyectan un extracto de código con la lógica de control del vestíbulo del IES El Caminàs que presenta una anomalía en un bucle:

```java
int totalLecturasValidas = 0;
int[] marcasHorarias = { 480, 482, 485, 495, 510 }; // Minutos desde medianoche (8:00 a 8:30)
int limiteRetraso = 485; // 8:05 h

for (int i = 0; i <= marcasHorarias.length; i++) {
    if (marcasHorarias[i] > limiteRetraso) {
        System.out.println("Acceso registrado con retraso.");
    } else {
        totalLecturasValidas++;
```

```
    }
}
```

Los estudiantes analizan el fragmento durante 3 minutos en su libreta técnica: 1. **Identificación de defectos:** - ¿Qué excepción arrojará la JVM en la última iteración del bucle? Un error de desbordamiento: `ArrayIndexOutOfBoundsException` provocado por la condición de parada incorrecta `i <= marcasHorarias.length` en lugar de `i < marcasHorarias.length`. 2. **Trazado de variables en tabla mental:** - ¿Qué valor acumulado tiene `totalLecturasValidas` en el instante inmediatamente anterior a que el programa colapse? 3. **Puesta en común:** Un estudiante voluntario marca en la pizarra en qué línea colocaría el punto de ruptura estratégico para interceptar el error antes de que la aplicación aborte la ejecución.

Sesión 2. Dojo de entrenamiento y katas de herramientas

Pasamos al dojo de herramientas. Afrontaremos tres **katas de depuración interactiva, control de ejecución paso a paso e inspección de memoria** sobre el proyecto propio de cada equipo, trabajando la sinergia directa con las estructuras algorítmicas de control:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). DIAGNÓSTICO CON PUNTOS DE RUPTURA E INSPECCIÓN DE VARIABLES EN TU PROYECTO PROPIO

- **Objetivo.** Iniciar tu proyecto bajo el control del depurador, fijar puntos de ruptura estratégicos en bucles o bloques condicionales y auditar el valor de los datos almacenados en las variables locales desde la pestaña **Variables** de IntelliJ IDEA sin recurrir a impresiones por consola.
- **Aplicación según tu proyecto de la bolsa:**
- En **Aventura conversacional**. Colocar un punto de ruptura al inicio del bucle del menú de decisiones del jugador. Verificar en el panel *Variables* cómo cambian los puntos de vida (`hp`) y el objeto seleccionado en el inventario antes de ramificar la escena.
- En **Motor de recomendación**. Situar el breakpoint en el bucle que recorre la matriz de valoraciones de ítems. Observar en tiempo real la variación del sumatorio de afinidad y el identificador de usuario activo en cada iteración.
- En **Simulador de físicas 2D**. Marcar un punto de ruptura dentro del bucle de cálculo temporal (`dt`). Comprobar la evolución de las componentes de velocidad `vx` , `vy` y las coordenadas `x` , `y` del cuerpo en colisión.
- En **Bóveda de contraseñas**. Poner el breakpoint en el bucle que valida la política de seguridad (longitud, caracteres especiales, mayúsculas). Constatar cómo cambian las banderas booleanas (`tieneMayuscula` , `tieneNumero`) al evaluar cada carácter.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). NAVEGACIÓN TÁCTICA (STEP OVER / STEP INTO) Y EVALUACIÓN DE EXPRESIONES AL VUELO (`Alt + F8`)

- **Contexto técnico.** Cuando un algoritmo falla en una rama condicional interna o en una llamada auxiliar, avanzar ciegamente es inútil. El desarrollador debe dominar la entrada y salida de funciones y calcular predicados lógicos sin reiniciar el software.
- **Misión de la kata:**

a. En una zona de bifurcaciones (`if` / `else` o `switch`), coloca un punto de ruptura justo en la cabecera de la condición.

b. Arranca en modo depuración (`Shift + F9`).

c. Utiliza **Step Over (`F8`)** para seguir el avance secuencial. Al llegar a una invocación a un método de cálculo o validación, pulsa **Step Into (`F7`)** para zambullirte en su interior y comprobar qué parámetros recibe.

d. Una vez verificada la primera línea, utiliza **Step Out (`Shift + F8`)** para retornar al flujo principal limpiamente.

e. En una línea con variables numéricas o booleanas, abre la herramienta de evaluación interactiva mediante `Alt + F8` **(Evaluate Expression)**.

f. Escribe en el cuadro de diálogo una consulta en vivo (por ejemplo, comprobar si una cadena cumple cierta condición o calcular una operación aritmética con las variables actuales). Verifica que IntelliJ evalúa y devuelve el resultado exacto consultando directamente la RAM del hilo en pausa.

g. Convierte uno de tus breakpoints en **Breakpoint condicional** (clic derecho sobre el punto rojo) para que la ejecución solo se congele cuando el contador de iteraciones supere un valor determinado (por ejemplo, `i == 3` o `reintentos > 2`).

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). MUTACIÓN DE VARIABLES EN MEMORIA EN CALIENTE (*SET VALUE*) Y DEPURACIÓN DE EXCEPCIONES SIN TOCAR CÓDIGO

- **Contexto de arquitectura.** Reproducir fallos intermitentes de red, límites de desbordamiento o entradas anómalas en el cliente puede requerir horas de simulación. Un desarrollador senior altera el estado interno de la memoria durante la propia ejecución para someter al software a pruebas de estrés inmediatas.
- **Misión de la kata:**

a. Pausa tu programa en un punto de ruptura antes de una bifurcación crítica (por ejemplo, la condición de si el usuario es administrador o si una cola de datos está llena).

b. En el panel **Variables**, haz clic derecho sobre la variable objetivo y selecciona **Set Value...** (o pulsa `F2`).

c. Modifica su valor directamente en la RAM (por ejemplo, cambia un valor `false` a `true`, o un identificador numérico por un límite negativo `-1`).

d. Pulsa `Intro` y avanza con `F8`: observa cómo el programa toma la rama lógica alternativa sin que hayas tenido que reescribir código ni recompilar el proyecto con Maven.

e. **Configuración de un Exception Breakpoint:**

- Abre el gestor de puntos de ruptura (`Ctrl + Shift + F8` o Menú *Run -> View Breakpoints...*).
- Pulsa el icono `+` (Add) en la esquina superior izquierda y selecciona **Java Exception Breakpoint**.
- Escribe en el buscador `java.lang.NullPointerException` (o la excepción que desees monitorizar) y confirma.
- Retira todos los demás puntos de ruptura de tu código y pulsa **Resume Program (`F9`)**.

f. Comprueba que, en caso de desencadenarse dicha excepción en cualquier rincón del código, la JVM se congelará de forma quirúrgica en la línea exacta que originó el fallo antes de abortar el programa, permitiéndote examinar la pila de llamadas (*Frames*) con total precisión.

4.1.2 Día 14 - 1 sesión

Sesión única. Taller práctico: Navegación por la pila de llamadas (*Call Stack*) y protocolo de informe técnico de incidencias (*Bug Report*)

1. Caso guía en AzaharTech

Es la mañana del miércoles 28 de octubre de 2026. En la sala de integración de **AzaharTech**, el desarrollador junior se encuentra frente a la pantalla con el semblante desencajado: la consola de IntelliJ IDEA acaba de escupir una cascada de más de treinta líneas de texto en color rojo brillante tras procesar un lote de prueba de carnés digitales del **IES El Caminàs**.

Pau Ferrer se acerca con una sonrisa comprensiva y apoya una taza de café sobre la mesa:

«No te asustes. El primer reflejo al ver una traza roja es cerrar la terminal o hacer scroll frenético buscando una palabra conocida. Los programadores novatos ven un error catastrófico; los profesionales vemos una autopsia perfecta.

*Ese bloque de texto no es ruido: es el mapa exacto del hilo de ejecución. Nos indica qué método llamó a qué otro método y en qué línea exacta la máquina virtual se quedó sin instrucciones válidas. Hoy aprenderemos a viajar hacia atrás en el tiempo utilizando el panel de marcos de ejecución (**Frames / Call Stack**) de IntelliJ y a redactar un informe formal de incidencia técnica. Porque en un equipo de consultoría como AzaharTech, un bug que no se puede reproducir ni documentar con rigor simplemente no existe».*

2. La Pila de Llamadas (*Call Stack*): Viajando entre dimensiones del código

Cuando un programa Java se ejecuta, la JVM gestiona la memoria de ejecución mediante una estructura de pila **LIFO** (*Last In, First Out*). Cada vez que un método invoca a otro, se apila un nuevo **marco de pila (*Stack Frame*)** que contiene sus

parámetros, variables locales y el contador de programa. Cuando el método termina con un `return`, su marco se destruye y el control regresa al nivel inmediatamente inferior.

```
FLUJO DE APILAMIENTO EN LA JVM (CALL STACK)

┌──────────────────────────────────────────┐ ▲  Cima de la pila
│  Frame 3: ValidadorQR.esFormatoValido()  │ │  (Lugar exacto del fallo)
├──────────────────────────────────────────┤ │
│  Frame 2: GestorAcceso.registrarEntrada()│ │  (Método intermedio)
├──────────────────────────────────────────┤ │
│  Frame 1: Main.main(String[] args)       │ │  (Punto de entrada)
└──────────────────────────────────────────┘ │

◄── Retorno del control / Destrucción del frame
```

NAVEGACIÓN INTERACTIVA EN EL PANEL *FRAMES* DE INTELLIJ

Cuando la ejecución está pausada en un punto de ruptura (o interceptada por un *Exception Breakpoint*): 1. **La columna izquierda de la pestaña Debug muestra los *Frames*:** En la parte superior aparece el método en el que estamos detenidos en este milisegundo; debajo, el método que lo invocó, y así sucesivamente hasta llegar al `main`. 2. **Viaje en el tiempo sin alterar el flujo:** Si haces clic en `Frame 2` o `Frame 1`, el editor de código saltará automáticamente a esa línea y el panel **Variables** se actualizará al instante para mostrar cómo estaban las variables en ese método superior, permitiéndote comprobar qué argumentos le envió al método fallido.

---

3. Estándar AzaharTech para el Registro de Incidencias Técnicas

En el ciclo de desarrollo profesional regulado por metodologías ágiles, cualquier error detectado durante las sesiones de depuración que no pueda subsanarse en el acto debe registrarse formalmente en el sistema de seguimiento de incidencias (*Issue Tracker* del repositorio Git).

Un informe técnico de incidencia (*Bug Report*) riguroso debe responder al siguiente estándar:

```
### [BUG] Desbordamiento de índice al escanear lote con identificador vacío

**1. Entorno de ejecución:**
- Versión del software: v0.1-sprint3 (commit `a4f82bc`)
- Entorno: OpenJDK 21 LTS / IntelliJ IDEA 2026.2 / Linux x86_64
- Módulo afectado: Núcleo de validación de accesos

**2. Descripción concisa:**
La aplicación aborta de forma inesperada cuando la cola de escaneo procesa un registro
cuyo campo de datos biométricos o identificador de matrícula contiene una cadena vacía.

**3. Pasos precisos para reproducir (Steps to Reproduce):**
1. Iniciar la aplicación en modo Debug con el dataset de prueba `test_alumnos_v1.csv`.
2. Introducir por consola la opción 2 ("Validación por lotes").
3. Asignar un tamaño de lote de 5 elementos incluyendo un registro sin identificador.

**4. Comportamiento observado (Actual Behavior):**
La JVM lanza una excepción `StringIndexOutOfBoundsException: Index 0 out of bounds for length 0`
y la aplicación finaliza abruptamente.

**5. Comportamiento esperado (Expected Behavior):**
El sistema debería capturar la anomalía, marcar el registro como inválido en el log
y continuar procesando los restantes alumnos de la cola sin abortar.

**6. Diagnóstico técnico y Call Stack:**
- Punto de interrupción: `ValidadorQR.java:38`
- Pila de llamadas:
    `at es.azahartech.caminas.ValidadorQR.comprobarPrefijo(ValidadorQR.java:38)`
    `at es.azahartech.caminas.GestorAcceso.procesarRegistro(GestorAcceso.java:74)`
    `at es.azahartech.caminas.Main.main(Main.java:21)`
- Inspección de memoria: Variable `qrData = ""` (longitud 0).
```

---

4. Dinámica de laboratorio: Simulación de fallo en cascada y auditoría cruzada

Durante los últimos 30 minutos de la sesión, cada estudiante pondrá a prueba la solidez de las estructuras de control que ha estado programando durante la semana:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      FASES DEL LABORATORIO PRÁCTICO (DÍA 14)             │
├──────────────┬──────────────────┬────────────────────────────────────────┤
│ Tiempo       │ Fase             │ Operación técnica requerida            │
├──────────────┼──────────────────┼────────────────────────────────────────┤
```

```
| Min 00 – 10  | 1. Provocación   | Forzar deliberadamente una excepción de ejecución en |
|              |    del fallo    | un método auxiliar profundo (por ejemplo, enviando   |
|              |                  | un parámetro nulo, una cadena vacía o índice fuera). |
|--------------|------------------|------------------------------------------------------|
| Min 10 – 20  | 2. Exploración   | Pausar con *Exception Breakpoint*, navegar hacia     |
|              |    de Frames     | atrás por los 3 primeros marcos en el panel *Frames* |
|              |                  | y localizar en qué línea se originó el dato corrupto.|
|--------------|------------------|------------------------------------------------------|
| Min 20 – 30  | 3. Redacción     | Crear el fichero `docs/incidencias/INCIDENCIA-01.md` |
|              |    del informe   | en el repositorio con la plantilla formal de bug.    |
└──────────────┴──────────────────┴──────────────────────────────────────────────┘
```

ADAPTACIÓN INMEDIATA SEGÚN EL PROYECTO DE LA BOLSA:

- **En Aventura conversacional:** Provocar un fallo al intentar acceder a la posición `[0]` de un inventario vacío dentro del método `consultarPrimerObjeto()`. Utilizar el panel *Frames* para retroceder hasta la llamada en `procesarAccionJugador()` y comprobar qué comando escribió el usuario.
- **En Motor de recomendación:** Provocar una división por cero (`ArithmeticException`) o un cálculo indefinido al promediar valoraciones de un usuario sin histórico. Rastrear en los marcos de pila qué filtro condicional falló en validar el divisor.
- **En Simulador de físicas 2D:** Forzar un parámetro de tiempo transcurrido negativo (`dt < 0`) y verificar a través de los *Frames* qué método de la física propagó el valor erróneo antes de que el motor actualizara las coordenadas de pantalla.
- **En Bóveda de contraseñas:** Generar un puntero nulo (`NullPointerException`) al intentar cifrar una contraseña que no ha sido inicializada en el formulario de registro. Inspeccionar el marco del método llamador para comprobar por qué el objeto llegó sin instanciar.

Al sonar el timbre, los estudiantes dejan confirmado su primer informe técnico de incidencia en Git. El código aún no es perfecto, pero el equipo dispone ya de los ojos de diagnóstico necesarios para abordar la construcción formal de casos de prueba y matrices de riesgo en las próximas semanas.

4.2 Semana 8. Clases de equivalencia y cobertura

4.2.1 Día 15 - 2 sesiones

Sesión 1. Teoría. La ingeniería del caso de prueba: cómo probar sin perderse en el infinito

1. Caso guía en AzaharTech

Lunes 2 de noviembre de 2026. Tras una primera semana de inmersión en el depurador interactivo, el equipo de primer curso de DAM regresa a las instalaciones de **AzaharTech** en Castellón de la Plana.

En la pantalla de la sala de reuniones, **Laia Claramunt** proyecta un formulario simple de la aplicación del **IES El Caminàs**: la ventana de registro de carnés temporales de visitante. El formulario solicita dos campos: la **edad del visitante** (de 12 a 99 años para acceder sin acompañante adulto) y un **código PIN de seguridad** de 4 dígitos (de `1000` a `9999`).

Laia mira a los estudiantes y lanza una pregunta directa:

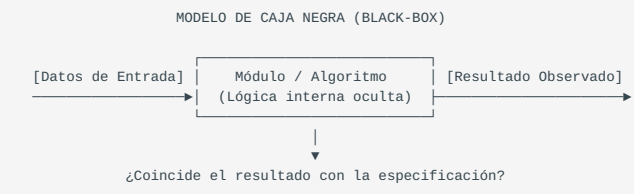
«Si quisiéramos demostrar matemáticamente que esta pequeña pantalla no tiene ningún fallo lógico, ¿cuántas combinaciones tendríamos que probar a mano?

*La edad admite teóricamente cualquier número entero de 32 bits en memoria (más de cuatro mil millones de valores), y el PIN tiene 9000 combinaciones válidas más infinitas cadenas alfanuméricas inválidas. Probar todas las entradas posibles por fuerza bruta (**exhaustion testing**) requeriría semanas de trabajo humano o miles de horas de computación inútil.*

En la industria del software no probamos al azar ni probamos todo: aplicamos técnicas formales de ingeniería para seleccionar el conjunto mínimo de datos que garantice la máxima detección de defectos. Hoy dejaremos de cazar bugs a ciegas y aprenderemos a diseñar casos de prueba profesionales utilizando dos pilares de las pruebas de caja negra: las clases de equivalencia y el análisis de valores límite».

2. Metodología de caja negra: Partición de Clases de Equivalencia

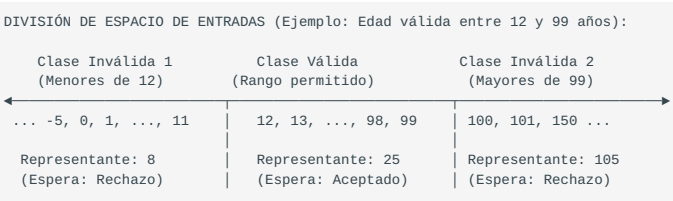
En las pruebas de **caja negra** (*Black-Box Testing*), nos centramos exclusivamente en las entradas y salidas del sistema según sus especificaciones funcionales, sin preocuparnos por cómo está escrito el código fuente por dentro.



PARTICIÓN EN CLASES DE EQUIVALENCIA (*EQUIVALENCE PARTITIONING*)

Esta técnica asume que el conjunto total de entradas posibles puede dividirse en grupos llamados **clases de equivalencia**. Si el programa trata correctamente un valor representativo de una clase, con alta probabilidad tratará igual a todos los demás valores de esa misma clase.

Para cada dato de entrada debemos identificar dos tipos de clases: 1. **Clases de equivalencia válidas:** Valores que cumplen las reglas de negocio y deben procesarse satisfactoriamente. 2. **Clases de equivalencia inválidas:** Valores incorrectos, fuera de rango o con formatos anómalos que el sistema debe rechazar de forma controlada (sin abortar el hilo de ejecución).



3. Análisis de Valores Límite (*Boundary Value Analysis - BVA*)

La experiencia en ingeniería del software demuestra que la inmensa mayoría de los defectos lógicos no ocurren en el centro de las clases de equivalencia (probar la edad `25` rara vez hace fallar a un algoritmo), sino **justo en los bordes de decisión**.

En Programación, los fallos en estructuras condicionales suelen deberse a confusiones sintácticas entre operadores: escribir `<` en lugar de `<=`, o arrancar un bucle en `0` en vez de `1` (*errores off-by-one*).

El **análisis de valores límite** complementa a las clases de equivalencia obligando a probar: * El valor mínimo y el valor máximo permitidos en la frontera. * Los valores inmediatamente adyacentes a cada frontera (justo por debajo y justo por encima).

MAPA DE VALORES LÍMITE (RANGO [12, 99])

Posición del límite	Valor probado	Tipo de clase	Resultado esperado
Límite inferior - 1	11	Inválida (demasiado joven)	Error: "Menor de edad no autorizado"
Límite inferior	12	Válida (mínimo legal)	Registro admitido
Límite inferior + 1	13	Válida (interior)	Registro admitido
Límite superior - 1	98	Válida (interior)	Registro admitido
Límite superior	99	Válida (máximo legal)	Registro admitido
Límite superior + 1	100	Inválida (edad fuera de rango operativo)	Error: "Edad excede el límite del sistema"

4. Estructura formal de un Caso de Prueba (*Test Case*)

En el estándar corporativo de AzaharTech, cada prueba formal de caja negra se documenta como un registro trazable con los siguientes atributos:

PLANTILLA DE ESPECIFICACIÓN DE CASO DE PRUEBA	
Identificador (ID)	CP-ACC-001 (Código unívoco vinculado al módulo funcional)
Nombre / Descripción	Validación de acceso en frontera inferior de edad permitida
Precondiciones	Base de datos del IES El Caminàs iniciada; terminal en espera
Datos de entrada	edad = 12; pin = 5540
Pasos de ejecución	1. Arrancar el validador. 2. Introducir valores. 3. Pulsar OK.
Resultado esperado	El sistema responde `ACCESO_CONCEDIDO` y genera un pase temporal
Criterio de éxito	Salida real idéntica al resultado esperado; sin excepciones JVM

5. Actividad activa interactiva: Caza de fronteras en la pizarra

Alba Torres plantea en la pizarra el requisito del identificador temporal de visitante para el IES El Caminàs:

«El sistema genera una clave de acceso temporal de 4 cifras numéricas. Para evitar códigos triviales de prueba de fábrica, el valor del PIN debe estar comprendido estrictamente entre `1000` y `9999` (ambos inclusive)».

Durante 5 minutos, los alumnos debaten y completan en la pizarra la tabla de valores críticos: 1. ¿Cuáles son los **valores límite** exactos que hay que probar? (`999` , `1000` , `1001` , `9998` , `9999` , `10000`). 2. ¿Qué ocurre con los casos especiales de frontera cualitativa? (por ejemplo, valor nulo `null` , cadena vacía `""` o texto con letras `"100A"`). 3. Conclusión guiada: *«Con solo 6 pruebas numéricas y 2 cualitativas cubrimos con rigor técnico el comportamiento de miles de combinaciones posibles»*.

Sesión 2. Dojo de entrenamiento y katas de herramientas

Nos trasladamos a los puestos de trabajo para ejecutar las katas prácticas de diseño formal de pruebas sobre los módulos que cada estudiante está construyendo en su proyecto individual:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). MATRIZ DE CLASES DE EQUIVALENCIA DE TU MÓDULO DE LÓGICA

- **Objetivo.** Tomar una función crítica de validación o cálculo de tu proyecto y definir formalmente en un documento Markdown (`docs/pruebas/matriz-equivalencia.md`) todas las clases válidas e inválidas posibles, asignando un dato representativo para cada una.
- **Aplicación según tu proyecto de la bolsa:**
- En **Aventura conversacional.** Analizar la función que procesa la compra de pociones en la tienda del juego. Parámetro: `oroDisponible` (debe ser mayor o igual al coste del ítem). Definir clases: saldo suficiente ($> coste$), saldo exacto ($== coste$), saldo insuficiente ($< coste$) y saldo negativo anómalo (< 0).
- En **Motor de recomendación.** Analizar la función de asignación de puntuaciones de afinidad (escala de 1 a 5 estrellas). Identificar clases válidas (rango entero $[1, 5]$) e inválidas (números < 1, números > 5, decimales o cadenas de texto no parseables).
- En **Simulador de físicas 2D.** Analizar la función que calcula el rebote elástico. Parámetro: coeficiente de restitución `e` (válido en el intervalo real $[0.0, 1.0]$). Definir clases válidas (rebote inelástico a elástico perfecto) e inválidas ($e < 0.0$ o $e > 1.0$).
- En **Bóveda de contraseñas.** Analizar la regla de longitud mínima y máxima para la contraseña maestra (longitud válida entre 8 y 32 caracteres). Definir clases válidas (longitud entre 8 y 32) e inválidas (longitud < 8, longitud > 32, cadena vacía y `null`).

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). BATERÍA FORMAL DE CASOS DE PRUEBA CON ANÁLISIS DE VALORES LÍMITE (BVA)

- **Contexto técnico.** Una vez delimitadas las clases de equivalencia, el desarrollador debe aterrizar los valores frontera en una tabla técnica de casos de prueba ejecutables, detallando entradas exactas y salidas esperadas.
- **Misión de la kata:**

a. Diseña una tabla en Markdown dentro de `docs/pruebas/casos-de-prueba.md` con un mínimo de **6 casos de prueba** derivados del análisis de valores límite de tu regla de negocio (tres para el límite inferior y tres para el límite superior).

b. Cada caso de prueba debe incluir: `ID`, `Entrada`, `Tipo de Límite (Inferior / Superior)`, `Resultado Esperado` y `Estado (Pendiente / Pasado / Fallido)`.

c. Ejecuta manualmente en tu programa cada uno de los casos diseñados introduciendo los valores por teclado o mediante llamadas en tu método `main`.

d. Anota con precisión quirúrgica el resultado obtenido. Si encuentras una discrepancia (por ejemplo, el programa acepta la longitud `7` cuando la especificación marcaba mínimo `8`), marca el caso como **Fallido** y no toques el código fuente todavía: el fallo ha sido formalmente acorralado.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). TRAZABILIDAD DE FRONTERA EN MEMORIA CON DEPURADOR Y ASERCIONES

- **Contexto de arquitectura.** Comprobar los valores límite por consola puede enmascarar si el programa superó la condición por el camino lógico correcto o por casualidad. En esta kata combinamos el depurador de la Semana 7 con la batería de pruebas de la Semana 8.
- **Misión de la kata:**

a. Abre el archivo Java donde reside la condición lógica evaluada en tu batería de pruebas.

b. Coloca un **punto de ruptura condicional** en la línea exacta del `if`. Configúralo para que se pause únicamente cuando la variable evaluada coincida con los valores límite críticos (por ejemplo: `longitud == 8 || longitud == 7`).

c. Lanza el programa en modo depuración (**Debug**, `Shift + F9`) y suministra el caso de prueba del límite conflictivo.

d. Cuando la ejecución se congele en la línea del `if`, abre la consola de evaluación (`Alt + F8`) y evalúa la expresión booleana completa de la condición.

e. Inspecciona paso a paso (`F8`) si el salto condicional entra por la rama verdadera o la rama falsa.

f. Si detectas un error de frontera (un clásico `<` en vez de `<=`), documenta el diagnóstico en la columna de observaciones de tu batería de pruebas, corrige el operador en el código fuente, recompila con Maven y vuelve a ejecutar la prueba para certificar que el caso cambia de **Fallido** a **Pasado**.

g. Haz commit en tu repositorio Git de los nuevos documentos de pruebas y de la corrección realizada bajo el mensaje: `fix(core): corregido error off-by-one en validacion de frontera segun CP-002`.

4.2.2 Día 16 - 1 sesión

Sesión única. Taller práctico: Medición del cubrimiento de código (*Code Coverage*) y procedimiento de ejecución de pruebas

---

1. Caso guía en AzaharTech

Miércoles 4 de noviembre de 2026. En el ecuador de la segunda semana del **Sprint 3**, el equipo de desarrollo se concentra en la sala de operaciones de **AzaharTech**.

Los estudiantes tienen sobre sus pantallas las baterías de casos de prueba diseñadas el lunes mediante partición de equivalencia y valores límite. **Alba Torres** se sitúa junto a la mesa del desarrollador junior y revisa el documento de pruebas del validador de accesos del **IES El Caminàs**:

«Habéis hecho un trabajo excelente acorralando los límites de entrada: el código ya no revienta con edades de 11 o 100 años ni con contraseñas vacías. Sin embargo, os hago una pregunta crítica de aseguramiento de la calidad (QA):

¿Estáis seguros de que vuestros casos de prueba han obligado a la JVM a pasar por todas las ramas de vuestro código? ¿O tenéis bloques de sentencias enteras dentro de los `else` o de los `catch` que jamás se han ejecutado en la realidad?

Si una línea de código no se ejecuta durante las pruebas, es código en la sombra: una bomba de relojería esperando a que un usuario real entre por ese camino en producción. Hoy utilizaremos el motor de cubrimiento de código (Code Coverage) integrado en IntelliJ IDEA para iluminar visualmente qué partes de nuestro software están blindadas y cuáles permanecen completamente a oscuras».

---

2. Fundamentos del cubrimiento de código (*Code Coverage*)

El **cubrimiento de código** es una métrica de **caja blanca** (*White-Box Testing*) que cuantifica el porcentaje de código fuente ejecutado cuando se somete una aplicación a una batería de pruebas.

No todas las métricas de cobertura ofrecen el mismo nivel de garantía:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                        NIVELES DE CUBRIMIENTO DE CÓDIGO                      │
├──────────────────────┬──────────────────────────────────┬────────────────────┤
│ Métrica              │ ¿Qué mide exactamente?           │ Nivel de rigor técnico │
├──────────────────────┼──────────────────────────────────┼────────────────────┤
│ 1. Cubrimiento de    │ Porcentaje de líneas de código   │ BÁSICO: Garantiza que la JVM │
│    sentencias/líneas │ que han sido ejecutadas al       │ ha pisado la instrucción, pero │
│    (Line Coverage)   │ menos una vez.                   │ ignora si se probaron todos los│
│                      │                                  │ caminos de una decisión.       │
├──────────────────────┼──────────────────────────────────┼────────────────────┤
│ 2. Cubrimiento de    │ Porcentaje de ramas evaluadas en │ MEDIO-ALTO: En un `if (A)`,  │
│    ramas/decisiones  │ estructuras condicionales        │ comprueba que se han probado   │
│    (Branch Coverage) │ (`if`, `switch`, bucles).        │ tanto el camino TRUE como el   │
│                      │                                  │ camino FALSE.                  │
├──────────────────────┼──────────────────────────────────┼────────────────────┤
│ 3. Cubrimiento de    │ Porcentaje de todas las          │ EXHAUSTIVO: En condiciones     │
│    condiciones       │ combinaciones booleanas          │ complejas `if (A && B)`, exige │
│    múltiples         │ posibles dentro de cada rama.    │ probar todas las combinaciones │
│                      │                                  │ de verdad de A y B.            │
└──────────────────────┴──────────────────────────────────┴────────────────────┘
```

```
ILUSTRACIÓN DE LA TRAMPA DEL CUBRIMIENTO DE LÍNEAS:

    if (esFinDeSemana && carnetCaducado) {   ◄── Línea evaluada
        denegarAcceso();                     ◄── Línea ejecutada (100% Line Coverage)
    }

    * Si solo probamos el caso (TRUE && TRUE), la cobertura de líneas dirá 100%.
    * Pero la cobertura de ramas alertará: "Falta probar qué ocurre cuando es FALSE".
```

Axioma de calidad de Laia Claramunt: *«Un cubrimiento de código del 100 % no demuestra que un programa carezca de fallos: no puede detectar requisitos que olvidaste programar. Sin embargo, un cubrimiento del 40 % demuestra matemáticamente que el 60 % de tu software es un territorio desconocido y sin control».*

---

3. El mapa visual de cobertura en IntelliJ IDEA

IntelliJ IDEA incorpora un motor nativo de instrumentación de bytecode que monitoriza cada instrucción ejecutada por la JVM.

CÓMO LANZAR LA EJECUCIÓN CON COBERTURA

En lugar del botón habitual de *Run* (`Shift + F10`) o *Debug* (`Shift + F9`), seleccionamos la opción **Run '...' with Coverage** (icono del triángulo verde superpuesto sobre un pequeño escudo de seguridad).

Al finalizar la ejecución, el editor de IntelliJ activa su semáforo de cobertura en el margen lateral izquierdo (*gutter*):

```
      CÓDIGO DE COLORES EN EL EDITOR DE INTELLIJ IDEA
```

```
┌─────┬──────────────────────────────────────────────────────┐
  Verde│   │ if (puntosVida > 0) {                              │ ◄── Rama completamente cubierta
       │   │     personaje.continuarAventura();                 │
       ├─────┼──────────────────────────────────────────────────┤
Amarillo│   │ if (intentosRestantes > 0 && servidorActivo) {     │ ◄── Rama parcialmente cubierta
       │   │     reintentarConexion();                          │     (solo se probó una condición)
       ├─────┼──────────────────────────────────────────────────┤
   Rojo│   │ } else {                                           │
       │   │     notificarAdministradorEmergencia();            │ ◄── Código no ejecutado
       │   │     bloquearTerminal();                            │     (0% cubrimiento / Punto ciego)
       └─────┴──────────────────────────────────────────────────┘
```

Además, IntelliJ despliega la ventana acoplable **Coverage**, que desglosa el porcentaje exacto de cobertura por paquetes, clases, métodos y líneas de todo el proyecto.

---

4. Dinámica de laboratorio: Cierre de ángulos muertos (30 minutos)

Durante la segunda mitad de la sesión, los estudiantes auditan su propio software aplicando la técnica de **pruebas dirigidas por cobertura (*Coverage-Driven Testing*)**:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                      FASES DEL LABORATORIO DE COBERTURA (DÍA 16)              │
├──────────────┬──────────────────┬────────────────────────────────────────────┤
│ Tiempo       │ Fase             │ Tarea técnica requerida                    │
├──────────────┼──────────────────┼────────────────────────────────────────────┤
│ Min 00 – 10  │ 1. Ejecución con │ Lanzar la clase principal o módulo de validación con │
│              │    Coverage      │ **Run with Coverage**. Registrar el % inicial de     │
│              │                  │ cobertura de líneas en el cuaderno de bitácora.      │
├──────────────┼──────────────────┼────────────────────────────────────────────┤
│ Min 10 – 20  │ 2. Detección de  │ Inspeccionar los bloques marcados en **rojo** o      │
│              │    bloques rojos │ **amarillo**. Identificar qué rama lógica jamás fue  │
│              │                  │ alcanzada por los casos de prueba del Día 15.        │
├──────────────┼──────────────────┼────────────────────────────────────────────┤
│ Min 20 – 30  │ 3. Diseño del    │ Crear un nuevo caso de prueba específico para        │
│              │    caso de cierre│ forzar la ejecución del bloque rojo y conseguir que  │
│              │                  │ todo el margen del editor se ilumine en **verde**.   │
└──────────────┴──────────────────┴────────────────────────────────────────────┘
```

CASUÍSTICA SEGÚN EL PROYECTO DE LA BOLSA:

- **En Aventura conversacional:** Identificar si la rama que gestiona la muerte del jugador (`hp <= 0`) o el rechazo de comandos desconocidos en el `default` del `switch` está en rojo. Diseñar un caso de prueba con un comando inválido (ej. `"VOLAR"`) para teñir la rama de verde.
- **En Motor de recomendación:** Comprobar si el bloque de control que gestiona usuarios sin ninguna valoración previa (*cold start*) está desatendido. Introducir un vector de valoraciones vacío para verificar que el mensaje de cortesía se ejecuta correctamente.
- **En Simulador de físicas 2D:** Verificar si las ramas que controlan la parada por fricción total (`velocidad < 0.001f`) o el choque contra el techo del escenario están sin cobertura. Diseñar una prueba que simule una caída libre prolongada para activar los límites de parada.
- **En Bóveda de contraseñas:** Comprobar si la rama que captura cadenas con caracteres de escape o espacios en blanco quedó sin evaluar en el bucle de validación. Suministrar una contraseña de prueba con espacios intermedios (ej. `"Admin 2026*"`) para forzar la ejecución del rechazo.

ENTREGABLE DE CIERRE DE SESIÓN

Cada alumno formaliza el estado de sus pruebas en el archivo `docs/pruebas/informe-cobertura.md`:

```
### Registro de Ejecución y Cubrimiento de Pruebas (Sprint 3 - Semana 8)

- **Módulo auditado:** Validador de reglas de negocio
- **Cubrimiento inicial de líneas:** 68% (Ramas `else` y excepciones no cubiertas)
- **Ángulo muerto detectado:** La excepción de argumento inválido nunca se disparaba.
- **Acción correctora:** Se añadió el caso de prueba `CP-VAL-007` con entrada nula.
- **Cubrimiento final alcanzado:** 92% de líneas / 85% de ramas.
```

Con la ejecución confirmada y registrada en el control de versiones, el equipo dispone de un código verificado con métricas objetivas, sentando la base para las pruebas estructurales avanzadas y la depuración de algoritmos complejos de la próxima semana.

4.3 Semana 9. Complejidad ciclomática y regresión

4.3.1 Día 17 - 2 sesiones

Sesión 1. Teoría. Desmontando el laberinto: grafos de flujo, métrica de McCabe y depuración no invasiva

1. Caso guía en AzaharTech

Lunes 9 de noviembre de 2026. Arranca la tercera y última semana del **Sprint 3** en la sede de **AzaharTech** en Castellón de la Plana.

En los módulos troncales, la exigencia técnica se encuentra en su punto álgido: en Programación, los alumnos están implementando algoritmos con bucles anidados (`for` , `while`), menús ramificados con `switch` y filtros encadenados con múltiples operadores lógicos (`&&` , `||`).

Sobre la pizarra de la sala técnica, **Alba Torres** y **Pau Ferrer** han transcrito el método principal del módulo de cálculo de sanciones y retrasos acumulados del **IES El Caminàs**. El código ocupa casi setenta líneas repletas de bifurcaciones anidadas:

```
public String calcularEstadoAsistencia(int minutosRetraso, boolean esJustificado, int faltasPrevias, boolean esReincidente) {
    String estado;
    if (minutosRetraso > 0) {
        if (!esJustificado) {
            if (minutosRetraso > 30) {
                estado = "FALTA_GRAVE";
            } else {
                estado = "RETRASO_LEVE";
            }
            if (faltasPrevias >= 3 || esReincidente) {
                estado += "_CON_EXPEDIENTE";
            }
        } else {
            estado = "JUSTIFICADO";
        }
    } else {
        estado = "PUNTUAL";
    }
    return estado;
}
```

Laia Claramunt toma el rotulador y rodea el método:

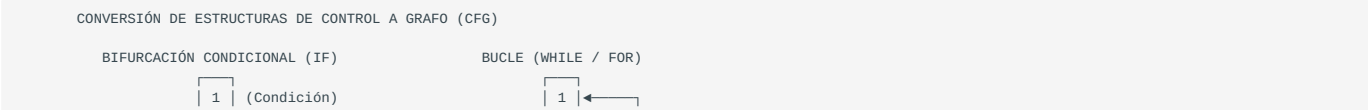
«Mirad este bloque. Cuando las condiciones se anidan, la mente humana tiende a colapsar: es el temido 'bloqueo algorítmico'. Es fácil perder la pista de qué camino toma la ejecución cuando se combinan variables booleanas y contadores numéricos.

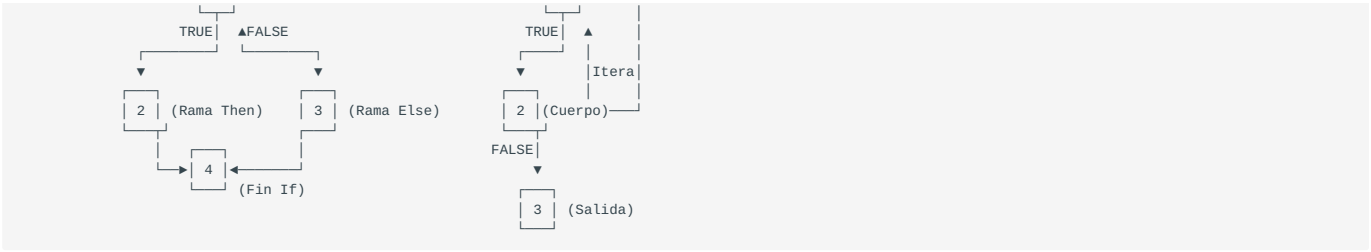
*A mediados de los años setenta, el matemático y científico de la computación Thomas McCabe formuló una regla que sigue siendo el estándar de oro en la ingeniería de software: la Complejidad Ciclomática.**

Hoy aprenderemos a traducir cualquier fragmento de código a un Grafo de Flujo de Control (CFG) para saber exactamente cuántos caminos independientes existen en un algoritmo. Y para no volvernos locos depurando bucles que se repiten cientos de veces, utilizaremos dos herramientas avanzadas de IntelliJ IDEA que no alteran el código: las variables monitorizadas (Watches) y los puntos de traza silenciosos (Logpoints)».

2. Grafos de Flujo de Control (CFG) y Complejidad Ciclomática de McCabe

El **Grafo de Flujo de Control** (*Control Flow Graph* o CFG) es una representación gráfica de todos los caminos que puede atravesar un programa durante su ejecución. * **Nodos (N):** Representan bloques de código secuencial (sin saltos ni bifurcaciones). * **Aristas (E):** Flechas dirigidas que representan la transferencia de control entre instrucciones.





LA FÓRMULA DE THOMAS MCCABE

La **Complejidad Ciclomática** ($V(G)$) cuantifica la complejidad lógica de un método y determina el número de **caminos linealmente independientes** que lo forman. Esto define el número mínimo de casos de prueba necesarios para garantizar que cada instrucción se ejecuta al menos una vez.

Se puede calcular de dos formas equivalentes: 1. **Fórmula topológica:** $$V(G) = E - N + 2P$$ *(Donde E es el número de aristas, N el de nodos y P el número de componentes conexos, normalmente $P = 1$ para un único método).* 2. **Regla de las regiones o puntos de decisión:** $$V(G) = \text{Número de puntos de decisión} + 1$$ *(Cada `if`, `while`, `for`, `case` o condición booleana compuesta con `&&` o `||` añade un punto de decisión).*

ESCALA DE RIESGO DE LA COMPLEJIDAD CICLOMÁTICA			
Valor V(G)	Nivel de riesgo	Mantenibilidad	Recomendación técnica
1 – 10	Código simple (Riesgo bajo)	Alta mantenibilidad Fácil de probar	Módulo bien estructurado. Diseño limpio y legible.
11 – 20	Complejidad moderada	Moderada dificultad de seguimiento	Requiere atención. Conviene extraer métodos auxiliares.
21 – 50	Alta complejidad (Riesgo alto)	Código inestable Propensión a bugs	ALERTA: Muy difícil de testear Refactorización prioritaria.
> 50	Inasumible (Riesgo crítico)	Intesteable "Código espagueti"	Código no aceptable según los estándares de AzaharTech.

---

3. Herramientas avanzadas del depurador para algoritmos complejos

Cuando un algoritmo ejecuta bucles de decenas o cientos de iteraciones, pulsar `F8` repetidamente es inviable. Para estos escenarios, IntelliJ IDEA ofrece dos herramientas de inspección no invasiva:

A) VARIABLES Y EXPRESIONES MONITORIZADAS (*WATCHES*)

En la pestaña **Variables** del panel Debug, podemos pulsar el icono `+` (**New Watch...**) para escribir expresiones dinámicas complejas que queramos vigilar constantemente (por ejemplo: `listaAlumnos.size()`, `i * factorEscala` o `persona != null && persona.esActivo()`). * A diferencia de inspeccionar una variable estática, el **Watch** reevalúa la fórmula matemática o lógica en cada micro-paso de la ejecución, destacando en color azul cualquier cambio de valor.

B) PUNTOS DE TRAZA SILENCIOSOS (*LOGPOINTS / TRACEPOINTS*)

Los **Logpoints** resuelven de raíz la tentación de meter sentencias `System.out.println` en el código fuente: 1. Coloca un punto de ruptura normal en el margen lateral. 2. Haz clic derecho sobre el punto rojo para abrir sus propiedades avanzadas. 3. **Desmarca** la casilla **Suspend** (para que la JVM **no detenga** la ejecución del programa). 4. **Marca** la casilla **Evaluate and log** (o *Breakpoint hit message*) y escribe una expresión Java entrecomillada o una variable:
`"Iteración: " + i + " | Minutos acumulados: " + totalMinutos` 5. Al ejecutar con Debug, IntelliJ imprimirá ese mensaje directamente en la consola de depuración cada vez que el hilo pase por esa línea, a máxima velocidad y sin haber modificado ni una sola coma del archivo fuente.

VENTANA DE CONFIGURACIÓN DE UN LOGPOINT EN INTELLIJ:

```
┌──────────────────────────────────────────────────────┐
│ Breakpoint: GestorAcceso.java:45                     │
├──────────────────────────────────────────────────────┤
│ [ ] Suspend   ◄── (Desmarcado: NO detiene el programa) │
│                                                      │
│ Log:                                                 │
│ [x] "Breakpoint hit" message                         │
```

```
| [x] Evaluate and log:                                    |
|     ┌──────────────────────────────────────────────┐     |
|     | "Alumno: " + id + " -> Estado: " + estado  |     |
|     └──────────────────────────────────────────────┘     |
└──────────────────────────────────────────────────────────┘
```

---

4. Actividad activa interactiva: Cálculo de caminos en la pizarra

Alba Torres retoma el método `calcularEstadoAsistencia` presentado al inicio. Los estudiantes analizan los puntos de decisión del código: 1. Condición 1: `if (minutosRetraso > 0)` (+1) 2. Condición 2: `if (!esJustificado)` (+1) 3. Condición 3: `if (minutosRetraso > 30)` (+1) 4. Condición 4: `if (faltasPrevias >= 3 || esReincidente)` (Contiene un operador `||`, sumando dos caminos booleanos: +2)

- **Cálculo:** $$\text{Puntos de decisión} = 1 + 1 + 1 + 2 = 5$$ $$V(G) = 5 + 1 = 6$$
- **Conclusión técnica:** *«Para tener una cobertura completa de caminos independientes de este método, necesitamos exactamente 6 casos de prueba distintos. Ni 5, ni 20: seis casos bien dirigidos garantizan que ningún recoveco quede sin auditar»*.

Sesión 2. Dojo de entrenamiento y katas de herramientas

Pasamos a los puestos de trabajo para aplicar las técnicas de análisis estructural y depuración avanzada sobre los algoritmos individuales de cada proyecto:

KATA 1 (CINTURÓN BLANCO / NIVEL BASE). GRAFO DE FLUJO Y CÁLCULO DE $V(G)$ DE TU ALGORITMO CENTRAL

- **Objetivo.** Seleccionar el método más complejo desarrollado hasta la fecha en tu proyecto propio (aquel que combine bucles y estructuras condicionales), dibujar su grafo de flujo de control elemental y calcular formalmente su Complejidad Ciclomática $V(G)$.
- **Aplicación según tu proyecto de la bolsa:**
- En **Aventura conversacional.** Analizar el método del bucle de combate o de resolución de acertijos (evaluación de vida del monstruo, tiradas de dados, estado de aturdimiento y huida).
- En **Motor de recomendación.** Analizar el algoritmo de ordenación o filtrado condicional por umbral de similitud (bucle que compara vectores de usuarios con condiciones múltiples).
- En **Simulador de físicas 2D.** Analizar el algoritmo de detección y respuesta de colisiones contra los cuatro bordes de la ventana y otros obstáculos rígidos.
- En **Bóveda de contraseñas.** Analizar la función de cálculo del nivel de entropía o fortaleza de contraseña (bucle que recorre la cadena comprobando caracteres contiguos repetidos, símbolos y números).
- **Entregable:** Registrar en `docs/pruebas/complejidad-algoritmo.md` el fragmento de código, el desglose de los puntos de decisión identificados y el valor resultante de $V(G)$.

KATA 2 (CINTURÓN MARRÓN / NIVEL AVANZADO). PRUEBAS DE CAMINO BÁSICO (*BASIS PATH TESTING*) E INSTRUMENTACIÓN CON LOGPOINTS

- **Contexto técnico.** Una vez determinado que un método tiene $V(G) = k$, debemos definir exactamente k caminos independientes que garanticen que cada arista del grafo se recorre al menos una vez, verificando su ejecución mediante *Logpoints*.

- **Misión de la kata:**

a. A partir del valor $V(G)$ obtenido en la Kata 1, define en Markdown la lista de los caminos básicos independientes enumerando los nodos que recorre cada uno (por ejemplo: *Camino 1: Inicio -> Entrada no válida -> Fin*; *Camino 2: Inicio -> Entrada válida -> Rama A -> Fin*, etc.).

b. En el archivo fuente Java, sustituye cualquier sentencia `System.out.println` de prueba que tuvieras por **Logpoints silenciosos** de IntelliJ IDEA (con la opción *Suspend* desmarcada).

c. Configura el mensaje del Logpoint para que imprima en cada iteración:
`"[TRACE] Camino activo: " + nombreVariableEstado + " | Valor: " + variableControl`

d. Ejecuta sucesivamente las entradas diseñadas para cada camino básico y comprueba en la pestaña **Debugger Console** cómo se generan las trazas limpias sin ensuciar la salida estándar del programa.

KATA 3 (CINTURÓN NEGRO / «HACKER AZAHARTECH»). DETECCIÓN DE BUCLES INFINITOS EN CALIENTE Y SALIDA FORZADA (*FORCE RETURN*)

- **Contexto de arquitectura.** El mayor riesgo de programar bucles complejos en el Sprint 3 es el bucle infinito: el programa deja de responder, la CPU se satura y el programador suele verse obligado a forzar el cierre violento del proceso perdiendo los datos de la sesión. Los desarrolladores senior interceptan y desarman bucles infinitos en caliente con el depurador.
- **Misión de la kata:**

a. En una rama experimental de tu proyecto, simula intencionadamente una condición de parada defectuosa en un bucle `while` (por ejemplo, omitiendo el incremento del contador o estableciendo una condición inalcanzable).

b. Arranca el programa en modo depuración (**Debug**, `Shift + F9`).

c. Introduce los datos para activar el bucle infinito: observa cómo la aplicación se queda congelada y no devuelve el control a la consola.

d. En la barra de herramientas del depurador de IntelliJ, pulsa el botón **Pause Program** (icono de dos barras verticales de pausa). La JVM suspenderá inmediatamente el hilo en la línea exacta del bucle donde se encuentra atrapado.

e. En el panel **Variables**, crea un **Watch** (`+`) con la condición de salida del bucle para entender por qué nunca se vuelve falsa.

f. **Maniobra de rescate (*Force Return*):**

- En el panel **Frames**, haz clic derecho sobre el marco del método actual atascado en el bucle.
- Selecciona la opción **Force Return**.
- Si el método devuelve un valor, introduce un retorno seguro (por ejemplo, `false`, `0` o `null`).

g. Observa cómo el depurador aborta la ejecución del bucle anómalo y devuelve el control al método llamador sin que la aplicación se caiga ni se corrompa el proceso de depuración.

h. Restaura la condición correcta en el código fuente, recompila con Maven y haz commit de las pruebas estructurales bajo el mensaje: `test(core): verificado conjunto de caminos basicos y complejidad ciclomática VG=X`.

4.3.2 Día 18 - 1 sesión

Sesión única. Taller de consolidación: El ciclo de vida del defecto (*Bug Lifecycle*), verificación de regresión y cierre formal del Sprint 3

---

1. Caso guía en AzaharTech

Viernes 13 de noviembre de 2026. Son las cuatro de la tarde en las oficinas de **AzaharTech** en Castellón de la Plana. La jornada representa un hito fundamental del primer trimestre: hoy concluye formalmente el **Sprint 3: «El ciclo del error: pruebas y depuración»**.

A lo largo de las últimas tres semanas, el equipo ha dado un salto cualitativo en su madurez técnica: ha dejado atrás la depuración artesanal basada en imprimir mensajes por consola para dominar el depurador interactivo de IntelliJ IDEA, la

inspección de variables en memoria RAM, el análisis de la pila de llamadas (*Frames*), el diseño de pruebas de caja negra (clases de equivalencia y valores límite), la medición de cubrimiento (*Coverage*) y el cálculo de la complejidad ciclomática de McCabe.

En la pantalla principal, **Laia Claramunt** y **Pau Ferrer** abren el tablero del proyecto del **IES El Caminàs**. Todos los módulos que la semana pasada provocaban bloqueos en los bucles o excepciones por valores fuera de rango se encuentran ahora corregidos.

Laia se dirige a los desarrolladores del equipo:

«Habéis transformado un código frágil e inestable en un producto predecible y blindado contra entradas inesperadas. Sin embargo, en el desarrollo profesional de software corporativo existe una regla inquebrantable: un fallo no está solucionado simplemente porque el programador que lo escribió diga 'en mi ordenador ya no falla'» [10, 12].

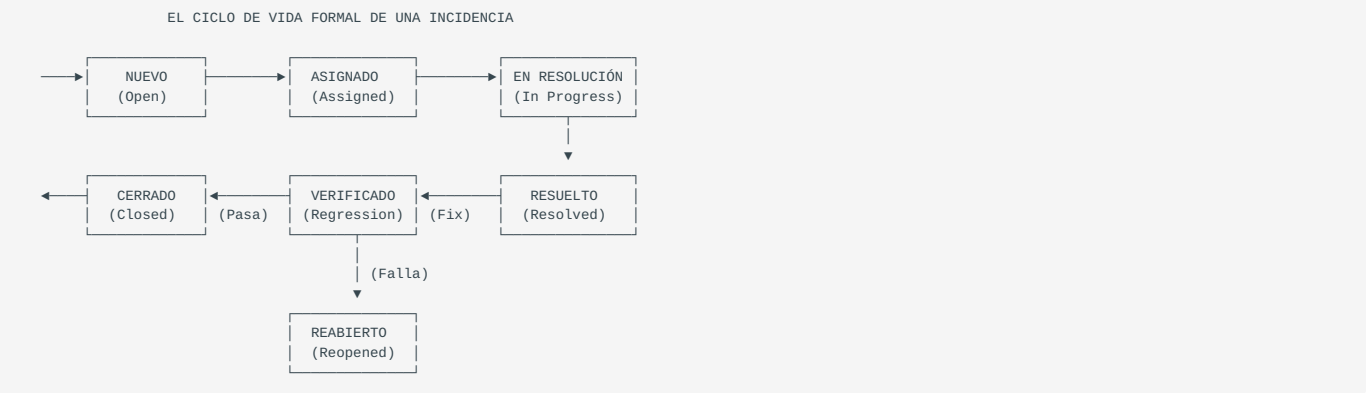
Para dar por cerrado un defecto, debemos someterlo a una prueba de regresión —garantizando que el arreglo no ha destruido accidentalmente una funcionalidad previa—, verificarlo mediante una auditoría técnica cruzada con otro compañero de consultoría y actualizar formalmente su estado en el ciclo de vida de la incidencia.

Hoy cerraremos el ciclo completo del Sprint 3: formalizaremos la matriz de trazabilidad de pruebas, auditaremos por parejas los módulos desarrollados y utilizaremos la interfaz gráfica de IntelliJ IDEA para realizar el commit y push visual definitivo que dejará el repositorio sellado antes de iniciar el lunes el Sprint 4: la programación orientada a objetos propia y la gestión de ramas en Git» [4, 11].

---

2. El Ciclo de Vida del Defecto (*Bug Lifecycle*) y las Pruebas de Regresión

En la ingeniería de software profesional regulada por estándares de calidad, un error o defecto técnico (*bug*) no es una simple anotación en una libreta: es una entidad con ciclo de vida propio que atraviesa estados unívocos desde que se detecta hasta que se certifica su corrección definitiva [3, 4].



LOS ESTADOS DE UNA INCIDENCIA TÉCNICA

1. **Nuevo (*Open*):** La anomalía ha sido detectada y registrada formalmente con su plantilla de informe técnico (*Bug Report*), detallando pasos para reproducirla y captura del comportamiento observado.
2. **Asignado (*Assigned*):** El equipo asigna la incidencia al desarrollador responsable de ese componente de software.
3. **En resolución (*In Progress*):** El desarrollador analiza el fallo con el depurador, identifica la causa raíz en memoria y modifica la lógica algorítmica.
4. **Resuelto (*Resolved*):** Se ha aplicado una corrección técnica en el código que supera el caso de prueba conflictivo.
5. **Verificado (*Verified / Re-test*):** Un segundo desarrollador (auditor de aseguramiento de calidad) reproduce de nuevo la prueba de forma independiente para certificar la corrección.
6. **Cerrado (*Closed*):** El defecto se archiva formalmente en el sistema documental tras superar las pruebas de regresión.
7. **Reabierto (*Reopened*):** Si la verificación falla o la corrección introduce nuevos fallos colaterales, la incidencia vuelve a asignarse al programador.

LAS PRUEBAS DE REGRESIÓN (*REGRESSION TESTING*)

Cuando se altera una línea de código para subsanar un fallo (por ejemplo, corregir un operador relacional `<` por `<=` en una condición de parada), existe siempre el peligro inminente de causar **regresión**: romper caminos lógicos o casos de uso que antes funcionaban a la perfección.

Una **prueba de regresión** consiste en volver a ejecutar de forma íntegra **toda** la batería de casos de prueba previamente aprobados tras aplicar el parche. Solo si se cumplen dos condiciones simultáneas se valida el cambio: * El nuevo caso de prueba que antes fallaba ahora se ejecuta con éxito. * Todos los casos de prueba históricos antiguos continúan pasando sin ninguna alteración ni degradación.

---

3. Matriz de Trazabilidad de Pruebas (*Traceability Matrix*)

Para certificar ante el cliente (en nuestro caso, el IES El Caminàs) que el software cumple los estándares contractuales, AzaharTech exige estructurar la **matriz de trazabilidad bidireccional**, vinculando cada Requisito Funcional (**RF**) con sus Casos de Prueba (**CP**) y sus Incidencias resueltas (**INC**) [2, 3]:

MATRIZ DE TRAZABILIDAD (SPRINT 3 - AZAHARTECH)				
Requisito	Descripción Funcional	Caso Prueba	Incidencia	Estado técnico
RF-CAM-01	Validación de carné de visitante [12-99]	CP-ACC-001 CP-ACC-002	INC-01 (Crash por nulo)	CERRADO y verificado
RF-CAM-02	Control horario de retraso lectivo	CP-HOR-001 CP-HOR-002	INC-02 (Bucle off-by-one)	CERRADO y verificado
RF-CAM-03	Notificación tras 3 faltas consecutivas	CP-SAN-001	Ninguna detectada	VERIFICADO (sin defectos)

---

4. Dinámica de laboratorio: Auditoría técnica cruzada por pares (30 minutos)

Durante la parte práctica de la sesión, los desarrolladores trabajan por parejas intercambiando sus puestos de trabajo para realizar una **Auditoría Técnica Cruzada (*Peer QA Review*)**:

FASES DE LA AUDITORÍA CRUZADA (DÍA 18)		
Tiempo	Fase	Operación técnica requerida
Min 00 – 10	1. Inspección de incidencias	Abrir en IntelliJ IDEA el proyecto del compañero. Abrir el archivo `docs/incidencias/INCIDENCIA-01.md`. Reproducir bajo depuración el fallo originalmente documentado para verificar el punto de ruptura.
Min 10 – 20	2. Prueba de regresión	Ejecutar toda la batería de casos de prueba del compañero con la herramienta **Run with Coverage**. Comprobar que no hay regresiones y que las líneas críticas se mantienen en verde.
Min 20 – 30	3. Firma del acta de cierre	Redactar conjuntamente el informe de cierre técnico en `docs/pruebas/cierre-sprint3.md` con las métricas finales consolidadas.

GUÍA DE VERIFICACIÓN SEGÚN EL PROYECTO DE LA BOLSA:

- **En Aventura conversacional:** El auditor introduce el valor de daño crítico exacto que reduce la vida del personaje a `0` puntos. Comprueba que el juego redirige a la escena de derrota limpiamente, sin excepciones de puntero nulo y confirmando que el inventario no se desborda en el bucle principal.
- **En Motor de recomendación:** El auditor ejecuta el cálculo de afinidad suministrando un vector de usuario con `0` valoraciones coincidentes. Verifica que el algoritmo maneja la división sin generar valores indefinidos (`NaN`) y que la lista ordenada final no se rompe.
- **En Simulador de físicas 2D:** El auditor simula una partícula impactando a velocidad máxima contra la esquina superior derecha (x_{max}, y_{max}). Confirma bajo el depurador que las dos componentes de aceleración y rebote se recalculan simultáneamente sin atrapar la simulación en un bucle infinito de colisión.

- **En Bóveda de contraseñas:** El auditor introduce una contraseña de prueba con la longitud máxima permitida (32 caracteres exactos). Verifica que el algoritmo de cifrado no trunca la cadena ni genera excepciones de desbordamiento de índice en memoria.

PLANTILLA DE CIERRE TÉCNICO

Cada pareja consolida el informe de calidad en el archivo `docs/pruebas/cierre-sprint3.md`:

```
### Acta de Cierre Técnico de Pruebas y Depuración (Sprint 3)

- **Desarrollador responsable:** [Nombre del Alumno]
- **Auditor de calidad asignado:** [Nombre del Compañero]
- **Estado de incidencias registradas:** 100% resueltas y verificadas (0 incidencias abiertas).
- **Métrica de cubrimiento alcanzada:** >= 85% de líneas / >= 80% de ramas.
- **Complejidad Ciclomática máxima auditada:** V(G) <= 6 (Módulo estructurado según estándares).
- **Resultado de pruebas de regresión:** SATISFACTORIO (Sin efectos colaterales detectados).
- **Dictamen técnico final:** APTO para integración en Sprint 4.
```

---

5. Cierre formal mediante Commit y Push visual en IntelliJ IDEA

En cumplimiento de las directrices metodológicas de AzaharTech para los primeros sprints, **todas las operaciones de control de versiones con Git se realizan exclusivamente a través del entorno gráfico de IntelliJ IDEA**, utilizándolo como un buzón visual de subida seguro y trazable [10]:

```
                VENTANA VISUAL DE COMMIT EN INTELLIJ IDEA (Ctrl + K)

┌──────────────────────────────────────────────────────────────────────────┐
│ Commit: Local Changes                                                    │
├──────────────────────────────────────────────────────────────────────────┤
│ Changes (3 files)                                                        │
│  [x] docs/pruebas/cierre-sprint3.md       (acta de cierre y auditoría) │
│  [x] docs/incidencias/INCIDENCIA-01.md    (estado cambiado a CERRADO)  │
│  [x] src/core/ValidadorLogica.java        (algoritmo blindado final)   │
├──────────────────────────────────────────────────────────────────────────┤
│ Mensaje de confirmación:                                                 │
│ ┌──────────────────────────────────────────────────────────────────────┐ │
│ │ docs(qa): auditoria cruzada completada y cierre formal del sprint 3 │ │
│ └──────────────────────────────────────────────────────────────────────┘ │
│                                                                          │
│ [ Commit v ]  ──► Desplegar flecha: [ Commit and Push... ]               │
└──────────────────────────────────────────────────────────────────────────┘
```

PASOS VISUALES PASO A PASO:

1. **Abrir el panel de confirmación:**

- Pulsa el atajo universal `Ctrl + K` (en Windows/Linux) o haz clic directo en la pestaña vertical izquierda **Commit** (`Alt + 0`).

2. **Auditoría visual de diferencias (*Diff Viewer*):**

- En la lista de archivos modificados (*Changes*), haz doble clic sobre `cierre-sprint3.md` y `ValidadorLogica.java`. Comprueba en el panel comparativo que las líneas agregadas coinciden con el acta y las correcciones de frontera.

3. **Selección de casillas de verificación:**

- Marca con un visto bueno (`[x]`) los archivos documentales y de código validados durante la sesión.

4. **Redacción del mensaje de commit:**

- En la caja de texto inferior escribe el mensaje estructurado de cierre: `docs(qa): auditoria cruzada completada y cierre formal del sprint 3`

5. **Confirmación y sincronización con GitHub (*Commit and Push*):**

- Haz clic sobre la pequeña flecha desplegable situada en el botón azul inferior de **Commit** y selecciona la opción **Commit and Push...** (o pulsa `Ctrl + Alt + K`).
- Se abrirá el cuadro de diálogo de confirmación de subida (*Push Commits*). Verifica que el destino sea tu repositorio remoto personal en GitHub y pulsa el botón azul **Push**.

Al aparecer la notificación flotante de confirmación en verde en la esquina inferior del IDE («*Pushed 1 commit to origin/main*»), el **Sprint 3: «El ciclo del error: pruebas y depuración»** queda oficialmente completado y auditado [10, 11]. El equipo está preparado para comenzar el próximo lunes el **Sprint 4**, donde nos adentraremos en el diseño de clases orientadas a objetos propias y el flujo de ramas de Git [4, 11].